

Contents			
1 Extra	1		
2 Data Structures	1		
2.1 BIT (Fenwick Tree)	1	4.11 Subset Sum	11
2.2 BIT (Fenwick Tree) 2D	1	4.12 Subset Sum with Bitset	11
2.3 DSU - Disjoint Set Union	1	5 Trees	11
2.4 DSU - Binary Tree	1	5.1 Centroid Decomposition	11
2.5 Implicit Treap	1	5.2 Sum of distances - rerooting	12
2.6 MinQueue	2	5.3 Edge HLD	12
2.7 Mo's Algorithm	2	5.4 HLD - Heavy light decomposition	12
2.8 Mo with updates	3	5.5 LCA - RMQ	13
2.9 Policy Based Data Structures	3	5.6 LCA - binary lifting	13
2.10 Segment Tree	3	6 Problemas clássicos	13
2.10.1 Iterative Point Update	3	6.1 2SAT	13
2.10.2 Iterative Lazy	4	6.2 Next Greater Element	14
2.10.3 Persistent	4	7 Strings	14
2.10.4 Recursive Lazy	4	7.1 Aho-Corasick	14
2.11 Sparse Table RMQ	5	7.2 Hashing	14
3 Graphs	5	7.3 KMP	14
3.1 Block Cut Tree (BCT)	5	7.4 Manacher	15
3.2 Bridges and Articulation points	5	7.5 Suffix Array	15
3.3 Dinic - Flow/matchings	6	7.6 Suffix Automaton	15
3.4 Euler path/tour - directed/undirected	6	7.7 Z	16
3.4.1 Euler path/tour - directed	6	8 Math	16
3.4.2 Euler path/tour - undirected	6	8.1 Combinatorics (Pascal's Triangle)	16
3.5 Floyd-Warshall	7	8.1.1 Combinatorial Analysis	16
3.6 Hopcroft-Karp - Bipartite Matching	7	8.2 Convolutions	17
3.7 Hungarian	7	8.2.1 AND convolution	17
3.8 Min cost flow	7	8.2.2 GCD convolution	17
3.9 SCC compressing	8	8.2.3 LCM convolution	17
3.10 SPFA	8	8.2.4 OR convolution	18
3.11 Steiner Tree	9	8.2.5 Subset Convolution	18
4 DP	9	8.2.6 XOR convolution	18
4.1 Bin Packing	9	8.3 Extended Euclid	18
4.2 Broken Profile DP	9	8.4 Factorization	18
4.3 Convex Hull Trick (CHT)	9	8.5 Gaussian elimination	18
4.4 Edit Distance (Levenshtein)	10	8.6 Inclusion-Exclusion Principle	18
4.5 Knapsack - 1D	10	8.7 Legendre's formula	19
4.6 Knapsack - 2D	10	8.8 Linear Sieve + Möbius Function	19
4.7 LCS - Longest Common Subsequence	10	8.8.1 Applications	19
4.8 LiChao Tree	10	8.9 Matrix template	19
4.9 LIS - Longest Increasing Subsequence	11	8.10 Miller-Rabin primality test	19
4.10 SOSDP	11	8.11 Mint	19
		8.12 Modular Inverse	20
		8.13 Number Theoretic Transform (NTT)	20
		8.13.1 NTT-Friendly Primes and Roots	20
		8.14 Pollard-Rho	20
		8.15 Euler's Totient	21
		8.16 Xor Basis	21
		9 Geometry	21
		9.1 Geometry	21
		1 Extra	
		2 Data Structures	
		2.1 BIT (Fenwick Tree)	
		Data structure that can handle point update and range queries for invertible operations. Obs: If the operation is not invertible (e.g. min,max,gcd), it can only answer prefix queries	
		<pre>1 // [l,r] interval convention and 0-indexed parameters 2 struct BIT { 3 vector<ll> bit; 4 int n; 5 BIT(int _n = 0){ 6 n = _n; 7 bit.assign(n+1,0ll); 8 } 9 ll presum(int i){ 10 ll ans = 0; 11 for (++i; i > 0; i -= i&-i) 12 ans += bit[i]; 13 return ans; 14 } 15 ll sum(int l, int r){ 16 return presum(r) - presum(l-1); 17 } 18 void add(int i, ll v){ 19 for (++i; i <= n; i += i&-i) 20 bit[i] += v; 21 } 22 int find_kth(int k) { 23 int pos = 0, sum = 0, step = 1; 24 while (step <= n) step <<= 1; 25 step >>= 1; 26 for (; step >= 1; step >>= 1) { 27 if (pos+step <= n && sum + bit[pos+step] <= k) { 28 pos += step; 29 sum += bit[pos]; 30 } 31 } 32 return pos; 33 } 34 };</pre>	
		2.2 BIT (Fenwick Tree) 2D	
		2D Sum BIT, update and sum. The struct encapsulates the 1-based indexing, therefore parameters are 0-based indexed.	

Query/update time: $\mathcal{O}(\log n \cdot \log m)$

Construction time (using proper constructor): $\mathcal{O}(nm)$

Space: $\mathcal{O}(nm)$

```
1 // [l,r] interval convention and 0-based indexing
2 struct BIT2D {
3     vector<ll> bit;
4     int n, m;
5     BIT2D(int _n, int _m){
6         n = _n; m = _m;
7         bit.resize((n+1)*(m+1));
8     }
9     BIT2D(vector<vector<ll>> &a) {
10        n = a.size();
11        m = a[0].size();
12        bit.resize((n+1)*(m+1));
13
14        for (int i = 1; i <= n; i++)
15            for (int j = 1; j <= m; j++)
16                bit[i*(m+1) + j] = a[i-1][j-1];
17
18        for (int i = 1; i <= n; i++) {
19            for (int j = 1; j <= m; j++) {
20                int nxt = j + (j&-j);
21                if (nxt <= m) bit[i*(m+1)+nxt] += bit[i*(m+1)+j];
22            }
23        }
24        for (int i = 1; i <= n; i++) {
25            int nxt = i + (i&-i);
26            if (nxt <= n) {
27                for (int j = 1; j <= m; j++)
28                    bit[nxt*(m+1) + j] += bit[i*(m+1) + j];
29            }
30        }
31    }
32    void add(int si, int sj, ll v){
33        for(int i = si+1; i <= n; i+=i&-i)
34            for (int j = sj+1; j <= m; j+=j&-j)
35                bit[i*(m+1) + j] += v;
36    }
37    ll presum(int si, int sj){
38        ll ans = 0;
39        for (int i = si+1; i > 0; i-=i&-i)
40            for (int j = sj+1; j > 0; j-=j&-j)
41                ans += bit[i*(m+1) + j];
42        return ans;
43    }
44    ll sum(ii ab, ii cd){
45        auto [a,b] = ab;
46        auto [c,d] = cd;
47        return presum(c,d) - presum(a-1,d) - presum(c,b-1)
48            + presum(a-1,b-1);
49    }
};
```

2.3 DSU - Disjoint Set Union

Query/update time: $\mathcal{O}(1)$

Construction time: $\mathcal{O}(n)$

Space: $\mathcal{O}(n)$

```
1 struct DSU {
```

```
2     vi p, sz;
3     DSU(int n) : p(n), sz(n,1) {
4         iota(p.begin(), p.end(), 0);
5     }
6     int find(int i) {
7         if (p[i] == i) return i;
8         return p[i] = find(p[i]);
9     }
10    bool join(int u, int v) {
11        u = find(u), v = find(v);
12        if (u == v) return false;
13        if (sz[u] < sz[v]) swap(u, v);
14        p[v] = u;
15        sz[u] += sz[v];
16        return true;
17    }
18 };
```

2.4 DSU - Binary Tree

Specific code to find maximum path sums between pairs of vertices. Uses Kruskal-style MST. Query/update time: possibly $\mathcal{O}(n)$ Construction time: $\mathcal{O}(n)$ Space: $\mathcal{O}(n)$

```
1 vi d;
2 vi_ii e;
3 vi ans;
4
5 int merged;
6 vi _p, _leaf, _wei;
7 vvi adj;
8 int _find(int u) { return _p[u] == u ? u : _p[u] = _find(_p[u]); }
9 void _union(int u, int v, int w){
10    u = _find(u);
11    v = _find(v);
12    int merge_ind = merged+n;
13    _p[u] = merge_ind;
14    _p[v] = merge_ind;
15    _leaf[merge_ind] = _leaf[u] + _leaf[v];
16    _wei[merge_ind] = max(_wei[u], _wei[v]);
17    adj[u].push_back(merge_ind);
18    adj[merge_ind].push_back(u);
19    adj[v].push_back(merge_ind);
20    adj[merge_ind].push_back(v);
21    merged++;
22 }
23 void make(){
24    _p = vi(2*n);
25    for(int i = 0; i < 2*n; i++) _p[i] = i;
26    _leaf = vi(2*n, 1);
27    _wei = vi(2*n);
28    for(int i = 0; i < n; i++) _wei[i] = d[i];
29    merged = 0;
30    adj = vvi(2*n);
31 }
32
33 void dfs(int u, int p){
34     for(auto &v: adj[u]){
35         if(v == p) continue;
36         ans[v] = ans[u] + (_leaf[u] - _leaf[v])*_wei[u];
37         dfs(v, u);
38     }
39 }
```

2.5 Implicit Treap

Randomized BST, handles insertion/deletion, reversals, cyclic shifts and more, all in $\mathcal{O}(\log n)$ per query. Can handle any associative function, just as a segtree can

```
1 mt19937 rng(1337); // change to chrono
2 struct implicit_treap {
3     struct tag {
4         int add = 0;
5         bool has = false;
6     };
7     struct node {
8         node *l = 0, *r = 0;
9         uint32_t y = rng();
10        int sz = 1;
11        bool rev = false;
12        // custom data
13        int val = 0, sum = 0;
14        tag lazy;
15
16        node(int _val) : val(_val), sum(_val) {}
17        void apply(tag t) {
18            if (!t.has) return;
19            val += t.add;
20            sum += t.add * sz;
21            lazy.add += t.add;
22            lazy.has = true;
23        }
24        void push() {
25            if (rev) {
26                swap(l, r);
27                if (l) l->rev ^= 1;
28                if (r) r->rev ^= 1;
29                rev = false;
30            }
31            if (lazy.has) {
32                if (l) l->apply(lazy);
33                if (r) r->apply(lazy);
34                lazy = tag();
35            }
36        }
37        void pull() {
38            sz = 1;
39            sum = val;
40            if (l) { sz += l->sz; sum += l->sum; }
41            if (r) { sz += r->sz; sum += r->sum; }
42        }
43    };
44    node* root = 0;
45    inline int size(node* t) { return t ? t->sz : 0; }
46    // split node t's first k indices
47    // write output treaps into L and R
48    void split(node* t, int k, node*& L, node*& R) {
49        if (!t) { L = R = 0; return; }
50        t->push();
51        if (size(t->l) >= k) {
52            split(t->l, k, L, t->l);
53            R = t;
54        } else {
55            split(t->r, k - size(t->l) - 1, t->r, R);
56            L = t;
57        }
58        t->pull();
59    }
60    // merge nodes L and R and write it into t
61    void merge(node*& t, node* L, node* R) {
62        if (!L || !R) { t = L ? L : R; return; }
63        L->push(); R->push();
```

```

64     if (L->y > R->y) {
65         merge(L->r, L->r, R);
66         t = L;
67     } else {
68         merge(R->l, L, R->l);
69         t = R;
70     }
71     t->pull();
72 }
73 implicit_treap(const vi& a) { // O(N) build
74     vector<node*> st; st.reserve(a.size());
75     for (int x : a) {
76         node *cur = new node(x), *last = 0;
77         while (!st.empty() && st.back()->y < cur->y) {
78             last = st.back(); st.pop_back();
79         }
80         cur->l = last;
81         if (!st.empty()) st.back()->r = cur;
82         st.push_back(cur);
83     }
84     root = st.empty() ? nullptr : st.front();
85     auto dfs = [&](auto &&dfs, node *t) -> void {
86         if (!t) return;
87         dfs(dfs, t->l);
88         dfs(dfs, t->r);
89         t->pull();
90     }; dfs(dfs, root);
91 }
92 void insert(int pos, int val) {
93     node *L, *R;
94     split(root, pos, L, R);
95     merge(L, L, new node(val));
96     merge(root, L, R);
97 }
98 void remove(int pos) {
99     node *L, *mid, *R;
100    split(root, pos, L, R);
101    split(R, 1, mid, R);
102    delete mid;
103    merge(root, L, R);
104 }
105 void rangeUpdate(int l, int r, tag v) {
106     node *L, *mid, *R;
107     split(root, l, L, R);
108     split(R, r - l + 1, mid, R);
109     if (mid) mid->apply(v);
110     merge(R, mid, R);
111     merge(root, L, R);
112 }
113 void reverse(int l, int r) {
114     node *L, *mid, *R;
115     split(root, l, L, R);
116     split(R, r - l + 1, mid, R);
117     if (mid) mid->rev ^= 1;
118     merge(R, mid, R);
119     merge(root, L, R);
120 }
121 int query(int l, int r) {
122     node *L, *mid, *R;
123     split(root, l, L, R);
124     split(R, r - l + 1, mid, R);
125     int ans = mid ? mid->sum : 0;
126     merge(R, mid, R);
127     merge(root, L, R);
128     return ans;
129 }
130 vi output() { // full array output
131     vi ans; ans.reserve(size(root));
132     auto go = [&](auto &&go, node *t) -> void {
133         if (!t) return;

```

```

134         t->push();
135         go(go, t->l);
136         ans.push_back(t->val);
137         go(go, t->r);
138     }; go(go, root);
139     return ans;
140 }
141 void erase_range(int l, int r) {
142     node *L, *mid, *R;
143     split(root, l, L, R);
144     split(R, r - l + 1, mid, R);
145     // care for mem leaks, destroy mid recursively if
146     // needed
147     merge(root, L, R);
148 }
149 void insert_range(int pos, node* new_treap) {
150     node *L, *R;
151     split(root, pos, L, R);
152     merge(L, L, new_treap);
153     merge(root, L, R);
154 };

```

2.6 MinQueue

Minimum (or maximum) queue. All operations are $\mathcal{O}(1)$ on average. Useful for sliding window max/min queries

```

1  template<typename T>
2  struct MinQueue {
3      deque<pair<T, int>> q;
4      int added = 0;
5      int removed = 0;
6      // returns [value, index]
7      pair<T, int> getmin() { return q.front(); }
8
9      void push(T x) {
10         // < for MaxQueue
11         while (!q.empty() && q.back().first > x) q.
12             pop_back();
13         q.push_back({x, added++});
14     }
15     void pop() {
16         if (!q.empty() && q.front().second == removed) q.
17             pop_front();
18         removed++;
19     }
20     bool empty() { return added == removed; }
21 };

```

2.7 Mo's Algorithm

A technique for solving offline range queries on static arrays by sorting queries to minimize total pointer movement. It processes intervals by incrementally updating the range via add and remove operations. With the optimal block size, the time complexity is $\mathcal{O}((N + Q)\sqrt{N})$ or $\mathcal{O}(N\sqrt{Q})$, depending on block size choice ($\sqrt{N}$ or $N/\sqrt{Q}$). This example solves queries for distinct elements in range

```
1 struct Mo {
```

```

2 struct Query {
3     int l, r, idx, b;
4     bool operator<(const Query& o) const {
5         return b != o.b ? b < o.b :
6             (b & 1 ? r > o.r : r < o.r);
7     }
8 };
9 int n, block_sz;
10 vi freq, a; // custom stuff
11 int ans = 0;
12 vector<Query> queries;
13 Mo(int n) : n(n), block_sz(round(sqrt(n))) {}
14 // [l,r] indexed
15 void add_query(int l, int r, int i) {
16     queries.push_back({l, r, i, l/block_sz});
17 }
18 void add(int i) { // add val at i
19     freq[a[i]]++;
20     if (freq[a[i]] == 1) ans++;
21 }
22 void remove(int i) { // remove value at i
23     freq[a[i]]--;
24     if (freq[a[i]] == 0) ans--;
25 }
26 int get_ans() { // compute current answer
27     return ans;
28 }
29 vi run() {
30     vi ans(queries.size());
31     sort(queries.begin(), queries.end());
32     int l = 0, r = -1;
33     for (auto& q : queries) {
34         while (l > q.l) add(--l);
35         while (r < q.r) add(++r);
36         while (l < q.l) remove(l++);
37         while (r > q.r) remove(r--);
38         ans[q.idx] = get_ans();
39     }
40     return ans;
41 }
42 };

```

2.8 Mo with updates

Also sorts queries to minimize total pointer movement, but now there is an added dimension for time (POINT updates). $\mathcal{O}(QN^{2/3})$

```

1 struct MoUpdate {
2     int n;
3     const int block_sz;
4     struct Query {
5         int l, r, t, id;
6     };
7     vector<Query> queries;
8     vector<ii> updates;
9     vi a;
10
11     MoUpdate(vector<int> &v, int q) : block_sz(round(cbrt
12         (2*v.size()*v.size()))) {
13         n = v.size() + q;
14         a = v;
15         init();
16     }
17     void add_query(int l, int r) {
18         queries.push_back({l, r, (int)updates.size(), (int)
19             queries.size()});

```

```

18 }
19 void add_update(int i, int x) {
20     updates.push_back({i,x});
21 }
22 void update(int l, int r, int t) {
23     auto &[i,v] = updates[t];
24     if (l <= i && i <= r) remove(i);
25     swap(a[i],v);
26     if (l <= i && i <= r) add(i);
27 }
28 vector<int> run() {
29     vector<int> ans(queries.size());
30     sort(queries.begin(), queries.end(), [&](Query qa,
31         Query qb){
32         int l = qa.l/block_sz;
33         int r = qa.r/block_sz;
34         int ql = qb.l/block_sz;
35         int qr = qb.r/block_sz;
36         using iii = tuple<int,int,int>;
37         return iii(l, l&1 ? -r:r, r&1 ? qa.t:-qa.t) <
38             iii(ql, ql&1 ? -qr:qr, qr&1 ? qb.t:-qb.t);
39     });
40     int l = 0, r = -1, t = 0;
41     for (auto &q : queries) {
42         while(t < q.t) update(l,r,t++);
43         while(t > q.t) update(l,r,--t);
44         while(l < q.l) add(--l);
45         while(r < q.r) add(++r);
46         while(l < q.l) remove(l--);
47         while(r > q.r) remove(r--);
48         ans[q.id] = get_ans();
49     }
50     return ans;
51 }
52 int ans = 0; // custom stuff
53 void init(){} // initialize custom stuff
54 void remove(int i) {} // remove a[i] from ans
55 void add(int i) {} // add a[i] to ans
56 int get_ans() {}
57 };
```

2.9 Policy Based Data Structures

Ordered Set and ordered multiset, possible to insert/erase and query index. All in $\mathcal{O}(\log n)$.

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 using ordered_set = tree<int,null_type,less<int>,
5     rb_tree_tag,
6     tree_order_statistics_node_update>;
7 // multiset: store {value, unique_id}
8 using ordered_multiset = tree<pair<ii,null_type,less<ii>,
9     rb_tree_tag,tree_order_statistics_node_update>;
10 // set:
11 s.insert(x); s.erase(x);
12 s.order_of_key(x); // # < x
13 *s.find_by_order(k); // k-th smallest (0-indexed)
14 // multiset:
15 ms.order_of_key({x,0}); // # < x
16 ms.order_of_key({x,INF}); // # <= x
17 ms.find_by_order(k)->first; // k-th value
```

2.10 Segment Tree

You should (almost) never alter the `segtree` logic. Ideally you only modify the `node` struct by answering four questions:

1. **tag**: What data does a range update carry? (e.g., a single integer for addition, or a pair of integers a, d for an AP).
2. **apply()**: How does a **tag** mathematically alter the state of a single node?
3. **push()**: How does a parent node transfer its pending tag to its left and right children?
4. **combine()**: How do you calculate the state of a parent by merging the states of its children?

OBS: lazy segtrees must be called with `has=true`

2.10.1 Iterative Point Update

Best for: Standard point updates and range queries (e.g., Point Add + Range Sum).

- **Performance:** Best possible.
- **Memory:** Best possible: $2n$ allocation.
- **Use Case:** If no range updates, then use this.

```

1 // ITERATIVE - implemented for sum + point assignment
2 struct segtree {
3     struct node {
4         int sum = 0; // data
5         static node combine(const node& a, const node& b) {
6             return {a.sum + b.sum};
7         }
8         void apply(int v) {
9             sum = v;
10        }
11    };
12    int n;
13    vector<node> t;
14    void init(const vi& a) {
15        n = a.size();
16        t.assign(2*n, node());
17        for (int i = 0; i < n; i++) t[n + i].apply(a[i]);
18        for (int i = n-1; i > 0; i--) t[i] = node::combine(
19            t[i<<1], t[i<<1|1]);
20    }
21    void set(int p, int v) {
22        for (t[p += n].apply(v), p >= 1; p > 0; p >= 1) {
23            t[p] = node::combine(t[p<<1], t[p<<1|1]);
24        }
25    }
26    node query(int l, int r) {
27        node resL, resR;
28        for (l+=n, r+=n; l < r; l>>=1, r>>=1) {
29            if (l&1) resL = node::combine(resL, t[l++]);
30            if (r&1) resR = node::combine(t[--r], resR);
31        }
32        return node::combine(resL, resR);
33    };
```

2.10.2 Iterative Lazy

Best for: Index-independent range updates (e.g., Range Add, Range Set, Range Max, Range Flip).

- **Performance:** Noticeably faster than recursive lazy trees.
- **Index Blindness:** If the update depends on the range indexes themselves (such as AP addition), use the recursive lazy one.

```

1 // LAZY ITERATIVE - implemented for Hash + range
2 // assignment
3 // for reference, see DoubleHashing implementation
4 // always call rangeUpdate with true on tag
5 struct segtree {
6     struct node {
7         struct tag {
8             int val = 0; // make sure default values are
9             // correct
10            bool has = false;
11        };
12        Hash hash; // custom stuff
13        int len = 0;
14        tag lazy;
15        // combine custom stuff
16        static node combine(const node &a, const node &b){}
17        void apply(tag t){
18            if (!t.has) return;
19            // apply tag too custom stuff
20            lazy = t; // remember to upd lazy tag
21        }
22        void push(node &left, node &right){
23            if (!lazy.has) return;
24            left.apply(lazy);
25            right.apply(lazy);
26            lazy = tag();
27        }
28        void pull(const node &left, const node &right){
29            tag cur = lazy;
30            *this = combine(left,right);
31            apply(cur);
32        }
33    };
34    int n, h;
35    vector<node> t;
36    segtree(const string& a) {
37        int sz = a.size();
38        n = 1; h = 0;
39        while (n < sz) { n*=2; h++; }
40        t.assign(2*n, node());
41        for (int i = 0; i < sz; i++) {
42            t[i+n].len = 1; // init custom stuff
43            t[i+n].hash.len = 1;
44            t[i+n].apply({a[i], true});
45        }
46        for (int i = n-1; i > 0; i--) t[i].pull(t[i<<1], t[
47            i<<1|1]);
48    }
49    void build(int p) {
50        while (p > 1) p >= 1, t[p].pull(t[p<<1], t[p
51            <<1|1]);
52    }
53    void push(int p) {
```

```

52     for (int s = h; s > 0; s--) {
53         int i = p >> s;
54         t[i].push(t[i<<1], t[i<<1|1]);
55     }
56 }
57 void rangeUpdate(int l, int r, node::tag v) {
58     if (l==r) return;
59     l += n, r += n;
60     int l0 = l, r0 = r;
61
62     push(l0); push(r0-1);
63     for (; l < r; l >= 1, r >= 1) {
64         if (l&1) t[l++].apply(v);
65         if (r&1) t[--r].apply(v);
66     }
67     build(l0); build(r0-1);
68 }
69 node query(int l, int r) {
70     if (l == r) return node();
71     l += n, r += n;
72     push(l); push(r-1);
73     node resL = node(), resR = node();
74     for (; l < r; l >= 1, r >= 1) {
75         if (l&1) resL = node::combine(resL, t[l++]);
76         if (r&1) resR = node::combine(t[--r], resR);
77     }
78     return node::combine(resL, resR);
79 }
80 };

```

2.10.3 Persistent

```

1  // PERSISTENT - implemented for sum + point assignment
2  struct persistent_segtree {
3      struct node {
4          int sum = 0;
5          int lc = 0, rc = 0; // indices to left and right
6          children
7          static node combine(const node& left, const node&
8              right) {
9              return {left.sum+right.sum};
10         }
11     };
12     void leaf(int v) { sum = v; }
13 };
14 int size;
15 vector<node> t;
16 vi root; // root node id for each version
17 persistent_segtree(const vi &a) {
18     size = 1;
19     while (size < a.size()) size <= 1;
20     t.push_back(node());
21     root.push_back(build(0, size, a));
22 }
23 int build(int lx, int rx, const vi &a) {
24     int x = t.size();
25     t.push_back(node());
26     if (rx-lx == 1) {
27         if (lx < a.size()) t[x].leaf(a[lx]);
28         return x;
29     }
30     int mx = (lx+rx)/2;
31     t[x].lc = build(lx, mx, a);
32     t[x].rc = build(mx, rx, a);
33     // do not override pointers lc,rc
34     t[x].sum = node::combine(t[t[x].lc], t[t[x].rc]).
35         sum;
36     return x;
37 }

```

```

34 int update(int i, int v, int old, int lx, int rx) {
35     int x = t.size();
36     t.push_back(t[old]);
37     if (rx-lx == 1) {
38         t[x].leaf(v);
39         return x;
40     }
41     int mx = (lx+rx)/2;
42     if (i < mx) t[x].lc = update(i, v, t[old].lc, lx,
43         mx);
44     else t[x].rc = update(i, v, t[old].rc, mx, rx);
45     // do not override pointers lc,rc
46     t[x].sum = node::combine(t[t[x].lc], t[t[x].rc]).
47         sum;
48     return x;
49 }
50 int update(int version, int i, int v) {
51     // assign to root[version] if this update
52     // should not trigger another version
53     int x = update(i, v, root[version], 0, size);
54     root.push_back(x);
55     return root.size()-1;
56 }
57 int copy_version(int version) {
58     root.push_back(root[version]);
59     return root.size() - 1;
60 }
61 node query(int l, int r, int x, int lx, int rx) {
62     if (lx >= r || rx <= l || x == 0) return node();
63     if (lx >= l && rx <= r) return t[x];
64     int mx = (lx+rx)/2;
65     return node::combine(
66         query(l, r, t[x].lc, lx, mx),
67         query(l, r, t[x].rc, mx, rx)
68     );
69 }
70 node query(int version, int l, int r) {
71     return query(l, r, root[version], 0, size);
72 }

```

2.10.4 Recursive Lazy

Best for: Index-dependent updates (e.g. AP addition), Tree Walking (Binary Search on Tree), and persistency.

- **Performance:** It is the slowest of the three, use only if necessary.

```

1  // LAZY recursive - implemented for sum + range AP
2  addition
3  struct segtree {
4      struct node {
5          int a = 0, d = 0; // be sure default is correct
6          bool has = false;
7      };
8      int sum = 0; // data
9      tag lazy;
10     static node combine(const node& a, const node& b) {
11         return {a.sum + b.sum, {0, 0, false}};
12     }
13     void leaf(int v) { sum = v; }
14     void apply(tag t, int lx, int rx) {
15         if (!t.has) return;
16         int len = rx - lx;
17
18         // Sum of (t.a + t.d * i) for i in [lx, rx-1]

```

```

19         sum += t.a * len + t.d*len*(lx+rx-1)/2;
20
21         lazy.a += t.a;
22         lazy.d += t.d;
23         lazy.has = true;
24     }
25     void push(node& left, node& right, int lx, int rx) {
26         if (lazy.has) {
27             int mx = (lx + rx) / 2;
28             left.apply(lazy, lx, mx);
29             right.apply(lazy, mx, rx);
30             lazy = tag();
31         }
32     }
33 };
34 int size;
35 vector<node> t;
36 void init(int n) {
37     size = 1;
38     while (size < n) size *= 2;
39     t.assign(2*size, node());
40     build(0, 0, size, n);
41 }
42 void build(int x, int lx, int rx, int n) {
43     if (rx-lx == 1) {
44         if (lx < n) t[x].leaf(0);
45         return;
46     }
47     int mx = (lx + rx) / 2;
48     build(2*x+1, lx, mx, n);
49     build(2*x+2, mx, rx, n);
50     t[x] = node::combine(t[2*x+1], t[2*x+2]);
51 }
52 void push(int x, int lx, int rx) {
53     t[x].push(t[2*x+1], t[2*x+2], lx, rx);
54 }
55 void rangeUpdate(int l, int r, node::tag v, int x,
56     int lx, int rx) {
57     if (lx >= r || rx <= l) return;
58     if (lx >= l && rx <= r) {
59         t[x].apply(v, lx, rx);
60         return;
61     }
62     push(x, lx, rx);
63     int mx = (lx + rx) / 2;
64     rangeUpdate(l, r, v, 2*x+1, lx, mx);
65     rangeUpdate(l, r, v, 2*x+2, mx, rx);
66     t[x] = node::combine(t[2*x+1], t[2*x+2]);
67 }
68 void rangeUpdate(int l, int r, int a, int d) {
69     rangeUpdate(l, r, {a - l*d, d, true}, 0, 0, size);
70 }
71 node query(int l, int r, int x, int lx, int rx) {
72     if (lx >= r || rx <= l) return node();
73     if (lx >= l && rx <= r) return t[x];
74     push(x, lx, rx);
75     int mx = (lx+rx) / 2;
76     return node::combine(
77         query(l, r, 2*x+1, lx, mx),
78         query(l, r, 2*x+2, mx, rx)
79     );
80 }
81 node query(int l, int r) {
82     return query(l, r, 0, 0, size);
83 }

```


2.11 Sparse Table RMQ

Sparse table for RMQ in $\mathcal{O}(1)$, used in many problems, including $\mathcal{O}(1)$ LCA (Trees) and LCP (SuffixArray) queries.

```
1 struct SparseTable {
2     vvi st;
3     SparseTable(const vvi &a) {
4         int n = a.size();
5         int max_log = __lg(n)+1;
6         st.assign(max_log, vector<ii>(n));
7         for (int i = 0; i < n; i++) {
8             st[0][i] = {a[i], i};
9         }
10        for (int i = 1; i < max_log; i++) {
11            for (int j = 0; j + (1 << i) <= n; j++) {
12                // Combine the two halves
13                st[i][j] = f(st[i-1][j], st[i-1][j+(1<<(i-1))]);
14            }
15        }
16    }
17    // Returns (min,id) in range [l, r] inclusive
18    ii min(int l, int r) {
19        int len = r - l + 1;
20        int k = __lg(len);
21        return f(st[k][l], st[k][r - (1<<k) + 1]);
22    }
23    inline ii f(ii l, ii r) { return std::min(l,r); }
24 };
```

3 Graphs

3.1 Block Cut Tree (BCT)

A data structure that condenses an undirected graph into a bipartite graph using two types of nodes: B-nodes (bi-connected components) and C-nodes (articulation points). Edges strictly connect a B-node to a C-node if the articulation point belongs to that component. Yields exactly one tree per connected component.

Articulation points are explicitly represented as the C-nodes. Because an articulation point is a vertex that disconnects a component upon removal, it acts as a literal bridge in the BCT. Any path navigating between two distinct biconnected components in the original graph is forced to route through the shared C-node.

```
1 vvi build_bct(const vvi &adj){
2     int n = adj.size();
3     vi in(n,-1), low(n), st;
4     int timer = 0;
5     vvi bccs;
6
7     auto dfs = [&](auto &&dfs, int u, int p) -> void {
8         in[u] = low[u] = ++timer;
9         st.push_back(u);
10        if (p == -1 && adj[u].empty()){
11            bccs.push_back({u});
12            st.pop_back();
```

```
13        return;
14    }
15    for (int v : adj[u]) if (v != p) {
16        if (in[v] != -1) {
17            low[u] = min(low[u], in[v]);
18        } else {
19            dfs(dfs,v,u);
20            low[u] = min(low[u],low[v]);
21            if (low[v] >= in[u]){
22                bccs.push_back({u});
23                while(1){
24                    int w = st.back();
25                    st.pop_back();
26                    bccs.back().push_back(w);
27                    if (w == v) break;
28                }
29            }
30        }
31    }
32 };
```

```
33 for (int i = 0; i < n; i++){
34     if (in[i] == -1) dfs(dfs,i,-1);
35 }
36 vvi bct(n+bccs.size());
37 for (int i = 0; i < bccs.size(); i++){
38     int b = n+i;
39     for (int u : bccs[i]){
40         bct[u].push_back(b);
41         bct[b].push_back(u);
42     }
43 }
44 return bct;
45 }
```

3.2 Bridges and Articulation points

DFS to get bridges and articulation points of a graph in $\mathcal{O}(n + m)$

```
1 vvi adj;
2 vi in, low;
3 int timer = 1; // start in 1
4 set<int> aps;
5 vector<ii> bridges;
6
7 void dfs_ap(int u, int p = -1) {
8     in[u] = low[u] = ++timer;
9     int ch = 0;
10    for (int v : adj[u]) {
11        if (v == p) continue;
12        if (in[v]) {
13            // Back-edge
14            low[u] = min(low[u], in[v]);
15        } else {
16            // Tree-edge
17            dfs_ap(v, u);
18            low[u] = min(low[u], low[v]);
19            if (low[v] >= in[u] && p != -1)
20                aps.insert(u);
21            ch++;
22        }
23    }
24    if (p == -1 && ch > 1)
25        aps.insert(u);
26 }
27
28 void dfs_bridges(int u, int p = -1) {
```

```
29     in[u] = low[u] = ++timer;
30     for (int v : adj[u]) {
31         if (v == p) continue;
32         if (in[v]) {
33             low[u] = min(low[u], in[v]);
34         } else {
35             dfs_bridges(v, u);
36             low[u] = min(low[u], low[v]);
37             if (low[v] > in[u])
38                 bridges.push_back({u, v});
39         }
40     }
41 }
```

3.3 Dinic - Flow/matchings

- **General Network:** $\mathcal{O}(VE \log U)$.
- **Unit Capacity Network:** $\mathcal{O}(\min(V^{2/3}, E^{1/2}) \cdot E)$. Often considered $\mathcal{O}(E\sqrt{V})$.
- **Bipartite Matching:** $\mathcal{O}(\min(V^{2/3}, E^{1/2}) \cdot E)$. Often considered $\mathcal{O}(E\sqrt{V})$.

```
1 struct Dinic {
2     struct Edge {
3         int u, v;
4         ll cap, flow = 0;
5         Edge(int u, int v, ll cap) : u(u), v(v), cap(cap) {}
6     };
7     const ll flow_inf = 1e18;
8     vector<Edge> edges;
9     vvi adj;
10    int n, m = 0;
11    int s, t;
12    vi level, ptr;
13    queue<int> q;
14    Dinic(int n): n(n) {
15        adj.resize(n);
16        level.resize(n);
17        ptr.resize(n);
18    }
19    void add_edge(int u, int v, ll cap) {
20        edges.emplace_back(u,v,cap);
21        edges.emplace_back(v,u,0);
22        adj[u].push_back(m++);
23        adj[v].push_back(m++);
24    }
25    bool bfs(ll delta){
26        queue<int> q;
27        q.push(s);
28        while(!q.empty()){
29            int u = q.front(); q.pop();
30            for (int id : adj[u]){
31                auto &e = edges[id];
32                if (e.cap - e.flow < delta) continue;
33                if (level[e.v] != -1) continue;
34                level[e.v] = level[u]+1;
35                q.push(e.v);
36            }
37        }
38        return level[t] != -1;
39    }
40    ll dfs(int u, ll pushed) {
```

```

41     if (pushed == 0) return 0;
42     if (u == t) return pushed;
43     for (int &cid = ptr[u]; cid < (int)adj[u].size();
44           cid++){
45         int id = adj[u][cid];
46         auto &e = edges[id];
47         if (level[u]+1 != level[e.v]) continue;
48         ll tr = dfs(e.v,min(pushed, e.cap - e.flow));
49         if (tr == 0) continue;
50         e.flow += tr;
51         edges[id^1].flow -= tr;
52         return tr;
53     }
54     return 0;
55 }
56 ll maxflow(int s, int t){
57     this->s = s; this->t = t;
58     ll max_c = 0;
59     for (auto &e : edges) max_c = max(max_c, e.cap);
60     ll delta = 1;
61     while(delta <= max_c) delta <= 1;
62     delta >= 1;
63     ll f = 0;
64     for (; delta > 0; delta >= 1){
65         while(true){
66             fill(level.begin(), level.end(), -1);
67             level[s] = 0;
68             if (!bfs(delta)) break;
69             fill(ptr.begin(), ptr.end(), 0);
70             while(ll pushed = dfs(s,flow_inf)) f += pushed;
71         }
72         return f;
73     }
74     // call constructor with (n1+n2+2) beforehand (dont
75     // add edges manually)
76     // assumes pairs are 1-indexed
77     vii maxmatchings(int n1, int n2, const vii& pairs){
78         for (int i = 1; i <= n1; i++)
79             add_edge(0,i,1);
80         for (int i = 1; i <= n2; i++)
81             add_edge(i+n1,n-1,1);
82         for (auto &[u,v] : pairs)
83             add_edge(u,v+n1,1);
84         maxflow(0,n-1);
85         vii matchings;
86         for (auto &e : edges){
87             if (e.u >= 1 && e.u <= n1 && e.flow == 1 && e.v >
88                 n1){
89                 matchings.emplace_back(e.u,e.v-n1);
90             }
91         }
92         return matchings;
93     }
94     vii mincut(int s, int t){
95         maxflow(s,t);
96         queue<int> q; q.push(s);
97         vector<bool> reachable(n);
98         reachable[s] = true;
99         while(!q.empty()){
100             int u = q.front(); q.pop();
101             for (auto &id : adj[u]){
102                 int v = edges[id].v;
103                 if (edges[id].cap - edges[id].flow > 0 && !
104                     reachable[v]) {
105                     reachable[v] = true;
106                     q.push(v);
107                 }
108             }
109         }
110     }

```

```

107     vii minCutEdges;
108     for (int i = 0; i < m; i += 2) {
109         const Edge& edge = edges[i];
110         if (reachable[edge.u] && !reachable[edge.v]) {
111             minCutEdges.emplace_back(edge.u, edge.v);
112         }
113     }
114     return minCutEdges;
115 }
116 };

```

3.4 Euler path/tour - directed/undirected

3.4.1 Euler path/tour - directed

```

1 auto get_path = [&](int s = -1, int t = -1) -> vi {
2     if (m == 0) return {};
3     if (s != -1 && t != -1){
4         if (out[s]-in[s] != 1 || out[t]-in[t] != -1) return
5         {};
6     }
7     int cnt1 = 0, cnt2 = 0;
8     for (int u = 0; u < n; u++) {
9         int d = out[u]-in[u];
10        if (d == 1) {
11            if (s == -1) s = u;
12            cnt1++;
13        } else if (d == -1) {
14            cnt2++;
15            if (t == -1) t = u;
16        } else if (in[u] != out[u]) return {};
17    }
18    if (!(cnt1==0 && cnt2==0) || (cnt1==1 && cnt2==1))
19        return {};
20    if (s == -1) {
21        for (int i = 0; i < n; i++) {
22            if (out[i] > 0) {
23                s = i; break;
24            }
25        }
26    }
27    vi path;
28    auto dfs = [&](auto& dfs, int u) -> void {
29        while (!adj[u].empty()) {
30            int v = adj[u].back();
31            adj[u].pop_back();
32            dfs(dfs, v);
33        }
34        path.push_back(u);
35    }; dfs(dfs, s);
36    if (path.size() != m+1) return {};
37    reverse(all(path));
38    return path;
39 };
40 vi path = get_path(0,n-1);
41 if (path.empty()) {
42     cout << "IMPOSSIBLE\n"; return;
43 }
44 for (int u : path) cout << u+1 << ' '; cout << '\n';

```

3.4.2 Euler path/tour - undirected

```

1 auto get_path = [&](int s = -1, int t = -1) -> vi {
2     if (m == 0) return {};
3     if (s != -1 && t != -1) {
4         if (s != t && (deg[s]%2 == 0 || deg[t]%2 == 0))

```

```

5         return {};
6         if (s == t && deg[s]%2) return {};
7     }
8     int odd = 0;
9     for (int u = 0; u < n; u++) {
10        if (deg[u]&1) {
11            odd++;
12            if (s == -1) s = u;
13            else if (t == -1 && s != u) t = u;
14        }
15    }
16    if (odd != 0 && odd != 2) return {};
17    if (s == -1) {
18        for (int u = 0; u < n; u++) {
19            if (deg[u] > 0) {
20                s = u; break;
21            }
22        }
23    }
24    if (odd == 2 && deg[s]%2 == 0) return {};
25    if (odd == 2 && deg[t]%2 == 0) return {};
26    vi path;
27    vector<bool> used(m, false);
28    auto dfs = [&](auto& dfs, int u) -> void {
29        while (!adj[u].empty()) {
30            auto [v, id] = adj[u].back();
31            adj[u].pop_back();
32            if (used[id]) continue;
33            used[id] = true;
34            dfs(dfs, v);
35        }
36        path.push_back(u);
37    }; dfs(dfs, s);
38    if (path.size() != m+1) return {};
39    reverse(all(path));
40    return path;
41 };
42 vi path = get_path(0,0);
43 if (path.empty()) {
44     cout << "IMPOSSIBLE\n"; return;
45 }

```

3.5 Floyd-Warshall

Time: $\mathcal{O}(n^3)$

```

1
2 vvi d(n, vi(n, INF));
3 void floyd_warshall(){
4     for (int k = 0; k < n; k++)
5         for (int i = 0; i < n; i++)
6             for (int j = 0; j < n; j++)
7                 d[i][j] = min(d[i][j], d[i][k]+d[k][j]);
8 }

```

3.6 Hopcroft-Karp - Bipartite Matching

Bipartite matching such as Kuhn but faster. BFS until first layer missing match, DFS for the BFS graph to find pairings. Time: $\mathcal{O}(E\sqrt{V})$

```

1 int n, m, k;
2 vvi adj;
3 vi p, dist; /* p is in matching for [0, n[ and parent
4               for [n, n+m[ */

```

```

4
5 int bfs(){
6     queue<int> q;
7     dist = vi(n+m, inf);
8     for(int i = 0; i < n; i++){
9         if(p[i] == -1) q.push(i), dist[i] = 0;
10    }
11    int min_dist_match = inf;
12    while(!q.empty()){
13        int u = q.front(); q.pop();
14        if(dist[u] > min_dist_match) continue;
15        for(auto v: adj[u]){
16            if(p[v] == -1) min_dist_match = dist[u];
17            else if(dist[p[v]] == inf){
18                dist[p[v]] = dist[u] + 1;
19                q.push(p[v]);
20            }
21        }
22    }
23    return min_dist_match != inf;
24 }
25
26 int dfs(int u){
27     for(auto v: adj[u]){
28         if(p[v] == -1 || (dist[u]+1 == dist[p[v]] && dfs(p[v]))){
29             p[v] = u;
30             p[u] = 1;
31             return true;
32         }
33     }
34     dist[u] = inf;
35     return false;
36 }
37
38 int hopkarp(){
39     p = vi(n+m, -1);
40     int matchings = 0;
41     while(bfs()){
42         for(int i = 0; i < n; i++){
43             if(p[i] == -1 && dfs(i)) matchings++;
44         }
45     }
46     return matchings;
47 }
48
49 void create(){
50     adj = vvi(n+m);
51     for(int i = 0; i < k; i++){
52         int u, v;
53         cin >> u >> v; u--; v--;
54         v += n;
55         adj[u].push_back(v);
56     }
57 }

```

3.7 Hungarian

Solves minimum cost assignment for n workers and m jobs.

Time: $\mathcal{O}((n+m)^3)$

```

1 // cost should be (cost[worker][job])
2 pair<int,vii> hungarian(int n, int m, const vvi &cost)
3 {
4     if (n == 0) return {0,{} };
5     int N = max(n, m);

```

```

6     vi u(N+1), v(N+1), p(N+1), way(N+1);
7
8     const int INF = 1e9;
9     for (int i = 1; i <= n; ++i) {
10        p[0] = i;
11        int j0 = 0;
12        vi minv(N + 1, INF);
13        vector<bool> used(N + 1, false);
14
15        do {
16            used[j0] = true;
17            int i0 = p[j0], delta = INF, j1;
18
19            for (int j = 1; j <= N; ++j) {
20                if (!used[j]) {
21                    int cur = cost[i0-1][j-1] - u[i0] - v[j];
22                    if (cur < minv[j]) {
23                        minv[j] = cur;
24                        way[j] = j0;
25                    }
26                    if (minv[j] < delta) {
27                        delta = minv[j];
28                        j1 = j;
29                    }
30                }
31            }
32
33            for (int j = 0; j <= N; ++j) {
34                if (used[j]) {
35                    u[p[j]] += delta;
36                    v[j] -= delta;
37                } else {
38                    minv[j] -= delta;
39                }
40            }
41
42            j0 = j1;
43        } while (p[j0] != 0);
44
45        do {
46            int j1 = way[j0];
47            p[j0] = p[j1];
48            j0 = j1;
49        } while (j0);
50    }
51
52    int total_cost = 0;
53    for (int j = 1; j <= m; ++j) {
54        if (p[j] != 0) {
55            total_cost += cost[p[j] - 1][j - 1];
56        }
57    }
58
59    // {worker, job}[] 0-indexed
60    vvi matchings;
61    for (int j = 1; j <= m; ++j) {
62        if (p[j] != 0) {
63            matchings.push_back({p[j] - 1, j - 1});
64        }
65    }
66    return {total_cost, matchings};
67 }

```

3.8 Min cost flow

Time: $\mathcal{O}(FE \log V)$

If negative costs are needed (maximize cost), need to run

SPFA once at the start, making the solution $\mathcal{O}(EV + FE \log V)$.

```

1 struct MinCostFlow {
2     struct Edge {
3         int to, cap, rev;
4         ll cost;
5     };
6     int n;
7     vector<vector<Edge>> adj;
8     MinCostFlow(int _n) : n(_n), adj(_n) {}
9     void add_edge(int u, int v, int cap, ll cost){
10        adj[u].push_back({v, cap, sz(adj[v]), cost});
11        adj[v].push_back({u, 0, sz(adj[u])-1, -cost});
12    }
13    //  $\mathcal{O}(FE \log V)$ 
14    lli min_cost_flow(int s, int t, int targetFlow) {
15        int flow = 0;
16        ll total_cost = 0;
17        vll dist, h(n);
18        vi pv, pe;
19        spfa(s, h, pv, pe); // only if < 0 costs exist
20        while (flow < targetFlow) {
21            djks(s, h, dist, pv, pe);
22            if (dist[t] == INF) break;
23            for (int i = 0; i < n; i++) {
24                if (dist[i] < INF) {
25                    h[i] += dist[i];
26                }
27            }
28            int f = targetFlow - flow;
29            int cur = t;
30            while (cur != s) {
31                f = min(f, adj[pv[cur]][pe[cur]].cap);
32                cur = pv[cur];
33            }
34            flow += f;
35            total_cost += f * h[t];
36            cur = t;
37            while (cur != s) {
38                Edge &e = adj[pv[cur]][pe[cur]];
39                e.cap -= f;
40                adj[e.to][e.rev].cap += f;
41                cur = pv[cur];
42            }
43        }
44        return {total_cost, flow};
45    }
46    // needed only if negative costs exists
47    void spfa(int s, vll &dist, vi &pv, vi &pe) {
48        dist.assign(n, INF);
49        pv = pe = vi(n, -1);
50        vector<bool> inq(n, false);
51        queue<int> q; q.push(s);
52        dist[s] = 0; inq[s] = true;
53        while (!q.empty()) {
54            int u = q.front(); q.pop();
55            inq[u] = false;
56            for (int i = 0; i < adj[u].size(); i++) {
57                Edge &e = adj[u][i];
58                int v = e.to;
59                if (e.cap > 0 && dist[v] > dist[u] + e.cost) {
60                    dist[v] = dist[u] + e.cost;
61                    pv[v] = u;
62                    pe[v] = i;
63                    if (!inq[v]) {
64                        inq[v] = true;
65                        q.push(v);
66                    }
67                }
68            }
69        }

```



```

68     }
69   }
70 }
71 void djc(int s, vll &h, vll &dist, vi &pv, vi &pe) {
72   dist.assign(n, INF);
73   pv = pe = vi(n, -1);
74   dist[s] = 0;
75   priority_queue<lli, vector<lli>, greater<lli>> pq;
76   pq.emplace(0, s);
77   while (!pq.empty()) {
78     auto [d, u] = pq.top(); pq.pop();
79     if (d > dist[u]) continue;
80     for (int i = 0; i < adj[u].size(); i++) {
81       Edge &e = adj[u][i];
82       if (e.cap <= 0) continue;
83       int v = e.to;
84       ll x = e.cost + h[u] - h[v];
85       if (dist[u] != INF && dist[v] > dist[u] + x) {
86         dist[v] = dist[u] + x;
87         pv[v] = u;
88         pe[v] = i;
89         pq.push({dist[v], v});
90       }
91     }
92   }
93 }
94 };
95 // usage
96 int nodes = 302; // amount of nodes in the network
97 MinCostFlow mcf(nodes);
98 for (int i = 0; i < 150; i++) {
99   mcf.add_edge(0, i+1, 1, 0); // source to node
100   mcf.add_edge(i+151, nodes-1, 1, 0); // node to sink
101 }
102 for (int i = 0; i < n; i++) {
103   int a, b, c; cin >> a >> b >> c;
104   mcf.add_edge(a, b+150, 1, -c); // edges in between (-
105   // c to maximize the cost)
106 }
107 // final max cost is -cost
108 auto [cost, flow] = mcf.min_cost_flow(0, nodes-1, 150);

```

3.9 SCC compressing

Condensing a graph into a DAG through its strongly connected components can be useful for DP

```

1 struct SCCCondenser {
2   int n, timer, scc_cnt;
3   vi in, low, scc;
4   stack<int> st;
5   SCCCondenser(const vvi& adj) {
6     n = adj.size();
7     in.assign(n, 0);
8     low.assign(n, 0);
9     scc.assign(n, -1);
10    timer = scc_cnt = 0;
11    for (int i = 0; i < n; ++i)
12      if (!in[i]) dfs(i, adj);
13  }
14  void dfs(int u, const vvi& adj) {
15    in[u] = low[u] = ++timer;
16    st.push(u);
17    for (int v : adj[u]) {
18      if (!in[v]) {
19        dfs(v, adj);
20        low[u] = min(low[u], low[v]);
21      } else if (scc[v] == -1) {

```

```

22        low[u] = min(low[u], in[v]);
23      }
24    }
25    if (low[u] == in[u]) {
26      while (true) {
27        int v = st.top(); st.pop();
28        scc[v] = scc_cnt;
29        if (u == v) break;
30      }
31      scc_cnt++;
32    }
33  }
34  // Returns {DAG, Aggregated Values}
35  pair<vvi, vi> build(const vvi& adj, const vi& val) {
36    vvi dag(scc_cnt);
37    vi scc_val(scc_cnt);
38    set<ii> edges;
39    for (int u = 0; u < n; ++u) {
40      scc_val[scc[u]] += val[u]; // Aggregate values
41      for (int v : adj[u]) {
42        if (scc[u] != scc[v]) {
43          if (edges.count({scc[u], scc[v]})) continue;
44          edges.insert({scc[u], scc[v]});
45          dag[scc[u]].push_back(scc[v]);
46        }
47      }
48    }
49    return {dag, scc_val};
50  }
51 };

```

3.10 SPFA

Detects negative cycles and generates them. If there are negative cycles, it sets the distances for vertices that do not have a shortest path to $-\text{INF}$ Time: $\mathcal{O}(VE)$

```

1 auto spfa = [&](int s, int n) -> pair<vi, vi> {
2   vi dist(n, INF), p(n, -1), cnt(n, 0);
3   vector<bool> inq(n, false), in_neg(n, false);
4   queue<int> q, neg_q;
5   vi cycle;
6   dist[s] = 0;
7   // if you need ANY neg cycle, push all vertices
8   // instead of a fixed source
9   q.push(s);
10  inq[s] = true;
11  while (!q.empty()) {
12    int u = q.front(); q.pop();
13    inq[u] = false;
14    for (auto [v, w] : adj[u]) {
15      if (dist[v] > dist[u] + w && !in_neg[v]) {
16        dist[v] = dist[u] + w;
17        p[v] = u;
18        cnt[v] = cnt[u] + 1;
19        if (cnt[v] >= n) {
20          if (cycle.empty()) {
21            int c = v;
22            for (int i = 0; i < n; i++) c = p[c];
23            for (int cur = c; ; cur = p[cur]) {
24              cycle.push_back(cur);
25              if (cur == c && cycle.size() > 1) break;
26            }
27            reverse(all(cycle));
28          }
29          if (!in_neg[v]) {
30            in_neg[v] = true;

```

```

31          neg_q.push(v);
32        }
33      } else if (!inq[v]) {
34        q.push(v);
35        inq[v] = true;
36      }
37    }
38  }
39  }
40  while (!neg_q.empty()) {
41    int u = neg_q.front(); neg_q.pop();
42    dist[u] = -INF;
43    for (auto [v, w] : adj[u]) {
44      if (!in_neg[v]) {
45        in_neg[v] = true;
46        neg_q.push(v);
47      }
48    }
49  }
50  // dist -INF is there is no shortest path
51  return {dist, cycle};
52 };

```

3.11 Steiner Tree

The Steiner Tree problem finds the minimum-cost tree connecting a given set of **terminals** in a weighted graph. The solution may use non-terminal vertices to reduce the total cost.

- **DP State:** $\text{dp}[\text{mask}][u] = \text{min cost to connect the terminals in mask ending at } u$.
- **Time:** $\mathcal{O}(3^S \cdot N + 2^S \cdot M \log N)$

Requires non-negative edge weights. Problem is NP-hard

```

1 int p = dsu.find(special[0]);
2 for (int i : special) {
3   if (dsu.find(i) != p) {
4     cout << "impossible\n"; return;
5   }
6 }
7 vvi dp(1<<s, vi(n, INF));
8 for (int i = 0; i < s; i++)
9   dp[1<<i][special[i]] = 0;
10
11 for (int mask = 0; mask < (1<<s); mask++) {
12   for (int sub = mask; sub > 0; sub = (sub-1)&mask) {
13     for (int u = 0; u < n; u++) {
14       dp[mask][u] = min(dp[mask][u], dp[sub][u] + dp[
15         mask^sub][u]);
16     }
17   }
18   priority_queue<ii, vii, greater<ii>> pq;
19   for (int u = 0; u < n; u++) pq.push({dp[mask][u], u});
20
21   while (!pq.empty()) {
22     auto [d, u] = pq.top(); pq.pop();
23     if (d > dp[mask][u]) continue;
24     for (auto [w, v] : adj[u]) {
25       if (dp[mask][v] > dp[mask][u] + w) {
26         dp[mask][v] = dp[mask][u] + w;
27         pq.push({dp[mask][v], v});
28       }
29     }
30   }

```

```

28     }
29 }
30 }

```

4 DP

4.1 Bin Packing

Time: $\mathcal{O}(n \cdot 2^n)$ Space: $\mathcal{O}(2^n)$

```

1  vi w(n);
2
3  vector<ii> dp(1<<n, ii(INF,0));
4  // dp[i] = for the subset i(bitmask) (A,B) is the pair
   where
5  // A - the min number of knapsacks to store this subset
6  // B - the min size of a used knapsack
7
8  dp[0] = ii(0,INF);
9  for (int subset = 1; subset < (1<<n); subset++){
10     for (int item = 0; item < n; item++){
11         if (!(subset>>item)&1) continue;
12         int prevsubset = subset - (1<<item);
13         ii prev = dp[prevsubset];
14
15         if (prev.second + w[item] <= x) {
16             // can fill the knapsack, fill it
17             dp[subset] = min(dp[subset], ii(prev.first, prev.
               second+w[item]));
18         } else {
19             // cant fill the knapsack, create a new one
20             dp[subset] = min(dp[subset], ii(prev.first+1, w[
               item]));
21         }
22     }
23 }
24
25 cout << dp[(1<<n)-1].first << endl;

```

4.2 Broken Profile DP

Solves the problem of counting how many ways to fill an $n \times m$ grid using 1×2 tiles. This technique can be used whenever the state dependence is only on the previous state (column). Time: $\mathcal{O}(mn2^n)$ Space: $\mathcal{O}(mn2^n)$

```

1  int dp[1002][12][1024];
2  dp[0][0][0] = 1;
3
4  for (int i = 0; i < m; i++){
5      for (int j = 0; j < n; j++){
6          for (int mask = 0; mask < (1<<n); mask++){
7              if (mask & (1<<j)){
8                  int nxt_mask = mask - (1<<j);
9                  dp[i][j+1][nxt_mask] += dp[i][j][mask];
10                 dp[i][j+1][nxt_mask] %= M;
11             } else {
12                 int q = mask + (1 << j);
13                 dp[i][j+1][q] += dp[i][j][mask];
14                 dp[i][j+1][q] %= M;
15                 if (j < n-1 && (mask & (1<<(j+1)))==0){
16                     q = mask + (1 << (j+1));

```

```

17         dp[i][j+1][q] += dp[i][j][mask];
18         dp[i][j+1][q] %= M;
19     }
20 }
21 }
22 }
23
24 for (int p = 0; p < (1<<n); p++){
25     dp[i+1][0][p] = dp[i][n][p];
26 }
27 }

```

4.3 Convex Hull Trick (CHT)

• Recurrence form:

$$dp_i = \min_{j < i} (dp_j + (h_i - h_j)^2 + c)$$

Expanding:

$$dp_i = h_i^2 + c + \min_{j < i} ((-2h_j)h_i + h_j^2 + dp_j)$$

Thus, for each j , insert the line

$$f_j(x) = (-2h_j)x + h_j^2 + dp_j$$

and query at $x = h_i$.

- **Slope monotonicity:** If coefficients a_j (slopes) are inserted in strictly decreasing (or increasing) order as j grows, and
- **Query monotonicity:** Values x_i for query come in non-decreasing (min) or increasing (max) order consistent with slope order,
- **Complexity:**
 - Insertion + amortized query in $\mathcal{O}(1)$ per operation (pointer walk) under monotonicity.
 - Non-monotonic case, generic CHT via binary search: $\mathcal{O}(\log n)$ per query.
 - General alternative: Li Chao Tree for insertion-queries in arbitrary order, $\mathcal{O}(\log M)$ per operation (where M is the domain of x).
- **Constraints:**
 - If it cannot be written in linear form, CHT does not apply.

– If there is no monotonicity of slopes or queries, consider Li Chao Tree or CHT variant with binary search.

The example below solves the dp where the recurrence is:

$$dp_i = \min_{j < i} (dp_j + (h_i - h_j)^2 + c).$$

```

1  struct CHT {
2      struct Line { // y = mx + c
3          int m, c;
4          Line(int m, int c) : m(m), c(c) {}
5          int val(int x){
6              return m*x + c;
7          }
8          int floorDiv(int num, int den) {
9              if (den < 0) num = -num, den = -den;
10             if (num >= 0) return num / den;
11             else return - ( (-num + den - 1) / den );
12         }
13         int ceilDiv(int num, int den) {
14             if (den < 0) num = -num, den = -den;
15             if (num >= 0) return (num + den - 1) / den;
16             else return - ( (-num) / den );
17         }
18         int intersect(Line l){
19             // m1x + c1 = m2x + c2
20             // x = (c2 - c1)/(m1 - m2)
21             // if slopes are increasing, use floor div
22             return ceilDiv(1.l.c - c, m - l.m);
23         }
24     };
25     deque<pair<Line, int>> dq;
26     void insert(int m, int c){
27         Line newLine(m, c);
28         if (!dq.empty() && newLine.m == dq.back().first.m)
29             {
30                 // If slopes increasing, change to <=
31                 if (newLine.c >= dq.back().first.c) return;
32                 else dq.pop_back();
33             }
34         // if slopes increasing, change to <=
35         while (dq.size() > 1 && dq.back().second >= dq.back()
36             .first.intersect(newLine)){
37             dq.pop_back();
38         }
39         if (dq.empty()){
40             // assuming queries are positive numbers, may
41             // change to -INF or +INF if needed
42             dq.emplace_back(newLine, 0);
43             return;
44         }
45         dq.emplace_back(newLine, dq.back().first.intersect(
46             newLine));
47     }
48     // dont use query and queryNonMonotonicValues in the
49     // same problem
50     int query(int x){
51         while (dq.size() > 1){
52             // if slopes increasing, change to >=
53             if (dq[1].second <= x) dq.pop_front();
54             else break;
55         }
56         return dq[0].first.val(x);
57     }
58     int queryNonMonotonicValues(int x){

```

```

54 int l=0, r=dq.size()-1, ans=0;
55 while (l <= r) {
56     int mid = (l+r)>>1;
57     if (dq[mid].second <= x) {
58         ans = mid;
59         l = mid + 1;
60     } else {
61         r = mid - 1;
62     }
63 }
64 return dq[ans].first.val(x);
65 }
66 };
67
68 void solve(){
69     int n, c; cin >> n >> c;
70     vi h(n), dp(n);
71     for (auto &x : h) cin >> x;
72     dp[0] = 0;
73     CHT cht;
74     cht.insert(-2*h[0], h[0]*h[0]);
75     for (int i = 1; i < n; i++){
76         dp[i] = cht.query(h[i]) + c + h[i]*h[i];
77         cht.insert(-2*h[i], h[i]*h[i] + dp[i]);
78     }
79     cout << dp[n-1] << endl;
80 }

```

4.4 Edit Distance (Levenshtein)

Very similar to LCS, in the sense that it considers prefixes already computed. Time: $\mathcal{O}(mn)$ Space: $\mathcal{O}(mn)$

```

1 vvi dp(n+1, vi(m+1));
2 for (int i = 0; i <= n; i++) dp[i][0] = i;
3 for (int i = 0; i <= m; i++) dp[0][i] = i;
4 for (int i = 1; i <= n; i++){
5     for (int j = 1; j <= m; j++){
6         dp[i][j] = min(
7             min(dp[i][j-1]+1, dp[i-1][j]+1),
8             dp[i-1][j-1] + (s[i-1]!=t[j-1])
9         );
10    }
11 }

```

4.5 Knapsack - 1D

Time: $\mathcal{O}(nW)$ Space: $\mathcal{O}(W)$

```

1 vi dp(W+1);
2 for (int i = 0; i < n; i++){
3     for (int j = W; j >= w[i]; j--){
4         dp[j] = max(dp[j], v[i] + dp[j-w[i]]);
5     }
6 }

```

4.6 Knapsack - 2D

Time: $\mathcal{O}(nW)$ Space: $\mathcal{O}(nW)$

```

1 vvi dp(n+1, vi(W+1));
2 for (int c = 1; c <= W; c++){

```

```

3     for (int i = 1; i <= n; i++){
4         dp[i][c] = dp[i-1][w];
5         if (c-w[i-1] >= 0) {
6             dp[i][c] = max(dp[i][c], dp[i-1][c-w[i-1]] + v[i-1]);
7         }
8     }
9 }

```

4.7 LCS - Longest Common Subsequence

Subsequence generation included here. Time: $\mathcal{O}(mn)$

Space: $\mathcal{O}(mn)$

```

1 vvi dp(n+1, vi(m+1));
2
3 for (int i = 1; i <= n; i++){
4     for (int j = 1; j <= m; j++){
5         if (a[i-1] == b[j-1]) {
6             dp[i][j] = dp[i-1][j-1]+1;
7         } else if (dp[i][j-1] > dp[i-1][j]){
8             dp[i][j] = dp[i][j-1];
9         } else {
10            dp[i][j] = dp[i-1][j];
11        }
12    }
13 }

```

4.8 LiChao Tree

Queries and insertions are all $\mathcal{O}(\log M)$. Where M is the size of the query interval the tree receives.

```

1 // Li Chao tree for min/max over domain [L, R].
2 // For int x use eps = 0 and discrete mid+1 split;
3 // For ld use eps>0 and split without +1.
4 template<typename T>
5 struct lichao_tree {
6     // if max lichao, change to ::min()
7     static const T identity = numeric_limits<T>::max();
8     struct Line {
9         T m, c;
10        Line() {
11            m = 0;
12            c = identity;
13        }
14        Line(T m, T c) : m(m), c(c) {}
15        T val(T x) {
16            if (c==identity) return identity;
17            return m * x + c;
18        }
19    };
20    struct Node {
21        Line line;
22        Node *lc, *rc;
23        Node() : lc(0), rc(0) {}
24    };
25    T L, R, eps;
26    deque<Node> buffer;
27    Node* root;
28    Node* new_node() {
29        buffer.emplace_back();
30        return &buffer.back();
31    }

```

```

32 lichao_tree() {}
33 lichao_tree(T _L, T _R, T _eps) {
34     init(_L, _R, _eps);
35 }
36 void clear() {
37     buffer.clear();
38     root = nullptr;
39 }
40 void init(T _L, T _R, T _eps) {
41     clear();
42     L = _L;
43     R = _R;
44     eps = _eps;
45     root = new_node();
46 }
47 void insert(Node* &cur, T l, T r, Line line) {
48     if (!cur) {
49         cur = new_node();
50         cur->line = line;
51         return;
52     }
53     T mid = l + (r-l) / 2;
54     if (r - l <= eps) return;
55     // if max lichao, change to >
56     if (line.val(mid) < cur->line.val(mid))
57         swap(line, cur->line);
58     // if max lichao, change to >
59     if (line.val(l) < cur->line.val(l)) insert(cur->lc,
60         l, mid, line);
61     else insert(cur->rc, mid + 1, r, line);
62 }
63 void insert_range(Node* &cur, int lx, int rx, int l,
64     int r, Line line) {
65     if (rx < l || r < lx || lx > rx || l > r) return;
66     if (!cur) cur = new_node();
67     if (l <= lx && rx <= r) return insert(cur, lx, rx,
68         line);
69     int mid = lx + (rx-lx)/2;
70     insert_range(cur->lc, lx, mid, l, r, line);
71     insert_range(cur->rc, mid+1, rx, l, r, line);
72 }
73 // insert a line that is only valid in range [l,r]
74 void insert_range(int m, int c, int l, int r) {
75     insert_range(root, L, R, l, r, Line(m, c));
76 }
77 T query(Node* &cur, T l, T r, T x) {
78     if (!cur) return identity;
79     T mid = l + (r-l) / 2;
80     T res = cur->line.val(x);
81     if (r - l <= eps) return res;
82     // if max lichao, change min to max
83     if (x <= mid) return min(res, query(cur->lc, l, mid,
84         x));
85     else return min(res, query(cur->rc, mid + 1, r, x));
86 }
87 void insert(T m, T c) { insert(root, L, R, Line(m, c)
88     ); }
89 T query(T x) { return query(root, L, R, x); }
90 };

```

4.9 LIS - Longest Increasing Subsequence

Time: $\mathcal{O}(n \log n)$

```

1 // dp[i] := min value subseq of len i+1 can end in
2 // dp is increasing sequence -> BS
3 int lis(const vi &a){

```

```

4  vi dp;
5  for (int x : a){
6      auto it = lower_bound(all(dp), x);
7      if (it == dp.end()) dp.push_back(x);
8      else *it = x;
9  }
10 return dp.size();
11 }

```

4.10 SOSDP

```

1  int k; // amount of bits
2  vi a(1<<k);
3  // sosdp
4  for (int bit = 0; bit < k; bit++){
5      for (int mask = 0; mask < (1<<k); mask++){
6          if (mask>>bit&1) {
7              a[mask] += a[mask ^ (1<<bit)];
8          }
9      }
10 }
11 // do stuff (such as multiplication for OR convolution)
12 // sosdp inverse
13 for (int bit = 0; bit < k; bit++){
14     for (int mask = 0; mask < (1<<k); mask++){
15         if (mask>>bit&1) {
16             a[mask] -= a[mask ^ (1<<bit)];
17         }
18     }
19 }

```

4.11 Subset Sum

Almost identical to Knapsack, this code contains the subset reconstruction. Time: $\mathcal{O}(nS)$ Space: $\mathcal{O}(nS)$

```

1  vvi dp(n+1, vi(sum+1));
2  vvii p(n+1, vii(sum+1));
3
4  dp[0][0] = 1;
5
6  for (int i = 1; i <= n; i++){
7      for (int s = 1; s <= sum; s++){
8          if (s-a[i-1] >= 0 && dp[i-1][s-a[i-1]]){
9              // sum is possible taking item i
10             p[i][s] = {i-1, s-a[i-1]};
11             dp[i][s] = 1;
12         } else if (dp[i-1][s]) {
13             // sum not possible taking item i
14             // but still possible with other items (<i)
15             p[i][s] = {i-1, s};
16             dp[i][s] = 1;
17         }
18     }
19 }
20 if (!dp[n][target]) {
21     cout << -1 << endl;
22     return;
23 }
24 vi subset;
25 ii pos = {n, target};
26 while(pos != ii(0,0)){
27     auto [i, s] = pos;
28     if (p[i][s].second != s) subset.push_back(a[i-1]);
29     pos = p[i][s];
30 }

```

4.12 Subset Sum with Bitset

If you only need reachability. Time: $\mathcal{O}(nS/w)$ (where w is sys word size) Space: $\mathcal{O}(S)$

```

1  vi a;
2  bitset<MAXSUM+1> dp; dp[0] = 1;
3  for (int x : a) dp |= (dp << x);

```

5 Trees

5.1 Centroid Decomposition

Recursively splits a tree by its center of mass, building a "Centroid Tree" with a strictly guaranteed $\mathcal{O}(\log N)$ depth. Used for path queries where the tree is unrooted.

Build: $\mathcal{O}(N \log N)$

Sample is solving "count distinct paths with number of edges between k_1 and k_2 "

```

1  struct CentroidDecomposition{
2      vvi t, ct;
3      vi sz, added, cp;
4      int n, root;
5      CentroidDecomposition(const vvi &adj, int _k1, int
6          _k2)
7      {
8          t = adj;
9          n = adj.size();
10         cp = sz = added = vi(n);
11         root = -1;
12         ct.resize(n);
13         init(_k1, _k2);
14         decompose(0, -1);
15     }
16     int dfs(int u, int p){
17         sz[u] = 1;
18         for (int v : t[u]){
19             if (v == p || added[v]) continue;
20             sz[u] += dfs(v, u);
21         }
22         return sz[u];
23     }
24     int centroid(int u, int size, int p = -1){
25         for (int v : t[u]){
26             if (v == p || added[v]) continue;
27             if (2*sz[v] > size) return centroid(v, size, u);
28         }
29         return u;
30     }
31     void decompose(int u, int p){
32         int size = dfs(u, p);
33         int c = centroid(u, size);
34         added[c] = 1;
35
36         if (p == -1) root = c;
37         else ct[p].push_back(c);
38         cp[c] = p;
39     }
40     process_centroid(c);

```

```

41     for (int v : t[c]){
42         if (!added[v])
43             decompose(v, c);
44     }
45
46     // custom stuff
47     int k1, k2;
48     ll ans;
49     vi freq, distfreq;
50     void init(int _k1, int _k2){
51         ans = 0;
52         k1 = _k1; k2 = _k2;
53         distfreq = freq = vi(k2+1);
54         freq[0] = 1;
55     }
56     void process_centroid(int c){
57         int maxd = 0;
58         int validsum = 0;
59         for (int u : t[c]) if (!added[u]){
60             distfreq[0] = 1;
61             int mxd = 0;
62             auto getdist = [&](auto &&getdist, int u, int p,
63                 int d) -> void {
64                 distfreq[d]++;
65                 mxd = max(mxd, d);
66                 if (d == k2) return;
67                 for (int v : t[u]){
68                     if (v == p || added[v]) continue;
69                     getdist(getdist, v, u, d+1);
70                 }
71             }; getdist(getdist, u, c, 1);
72             int sum = validsum;
73             for (int d = 1; d <= mxd; d++){
74                 ans += 1ll*sum*distfreq[d];
75                 sum -= freq[k2-d];
76                 if (k1-d-1 >= 0) sum += freq[k1-d-1];
77             }
78             for (int d = 1; d <= mxd; d++) {
79                 if (k1 <= d) validsum += distfreq[d];
80                 freq[d] += distfreq[d], distfreq[d] = 0;
81             }
82             maxd = max(maxd, mxd);
83         }
84         for (int i = 1; i <= maxd; i++) freq[i] = 0;
85     };

```

5.2 Sum of distances - rerooting

Given a tree, $f(u, v) :=$ distance from u to v in the tree, compute

$$\sum_{u,v} f(u, v)$$

. Time: $\mathcal{O}(n)$

```

1  vvi adj;
2  vi sum_down, sum_up, sz;
3  void dfs(int u, int p){
4      for (auto v : adj[u]){
5          if (v == p) continue;
6          dfs(v, u);
7          sz[u] += sz[v];
8          sum_down[u] += sum_down[v];
9      }
10     sum_down[u] += sz[u];

```

```

11 }
12 void dfs2(int u, int p, int par_ans){
13     int up_amount = sz[0] - sz[u];
14     sum_up[u] += par_ans + up_amount;
15     int sum = sum_down[u];
16     for (auto v : adj[u]){
17         if (v == p) continue;
18         // new p_ans is going up for is going up for p
19         // + go up to u and go down to other subtrees
20         dfs2(v,u, par_ans + (sz[0]-sz[v]) + sum - (sum_down
21             [v]+sz[v]));
22     }
23 }

```

5.3 Edge HLD

Sometimes the value is on the edges, for this few things need to change, but here is a template. Pre-computation: $\mathcal{O}(n)$ Queries: $\mathcal{O}(\log^2 n)$

```

1 struct EdgeHLD {
2     int n, timer = 0;
3     vvii adj;
4     vi par, dep, sz, heavy, head, val, tin, tout;
5     segtree seg;
6     void init(int _n, vvii& _adj) {
7         n = _n;
8         adj = _adj;
9         par = heavy = vi(n,-1);
10        val = dep = sz = head = tin = tout = vi(n);
11        timer = 0;
12        // edgeWeight[v] = weight of edge (par[v], v), for
13        // v>0
14        // root (0) has no par, so its val is (0)
15        dfs1(0,0,0);
16        dfs2(0, 0);
17        vi linear(n);
18        for (int u = 0; u < n; u++)
19            linear[tin[u]] = val[u]; // position stores edge
20            weight
21        seg.init(linear);
22    }
23    int dfs1(int u, int p, int w) {
24        sz[u] = 1;
25        par[u] = p;
26        val[u] = w;
27        int max_sz = 0;
28        for (auto [v,w] : adj[u]) {
29            if (v == p) continue;
30            dep[v] = dep[u] + 1;
31            int size = dfs1(v, u, w);
32            sz[u] += size;
33            if (size > max_sz) {
34                max_sz = size;
35                heavy[u] = v;
36            }
37        }
38        return sz[u];
39    }
40    void dfs2(int u, int h) {
41        tin[u] = timer++;
42        head[u] = h;
43        if (heavy[u] != -1)
44            dfs2(heavy[u], h);
45        for (auto [v,w] : adj[u]) {
46            if (v != par[u] && v != heavy[u])
47                dfs2(v, v);
48        }
49        tout[u] = timer;
50    }
51    void update(int u, int val) {
52        seg.set(tin[u], val);
53    }
54    void rangeUpdate(int u, int v, int x) {
55        while (head[u] != head[v]) {
56            if (dep[head[u]] < dep[head[v]]) swap(u, v);
57            seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
58            u = par[head[u]];
59        }
60        if (dep[u] > dep[v]) swap(u, v);
61        ans = seg.merge(ans, seg.query(tin[u], tin[v]+1));
62        return ans;
63    }
64    segtree::node query_subtree(int u) {
65        // query all edges in subtree of u
66        return seg.query(tin[u] + 1, tout[u]);
67    }
68 }

```

```

46 }
47 tout[u] = timer;
48 }
49 // u must be the child of the edge
50 void update_edge(int u, int val) {
51     seg.set(tin[u], val);
52 }
53 void rangeUpdate(int u, int v, int x) {
54     while (head[u] != head[v]) {
55         if (dep[head[u]] < dep[head[v]]) swap(u, v);
56         seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
57         u = par[head[u]];
58     }
59     if (dep[u] > dep[v]) swap(u, v);
60     seg.rangeUpdate(tin[u]+1, tin[v]+1, x);
61 }
62 void update_subtree(int u, int x) {
63     // updates all in subtree of u (edge p[u] -> u)
64     seg.rangeUpdate(tin[u] + 1, tout[u], x);
65 }
66 segtree::node query(int u, int v) {
67     segtree::node ans;
68     while (head[u] != head[v]) {
69         if (dep[head[u]] < dep[head[v]]) swap(u, v);
70         ans = seg.merge(ans, seg.query(tin[head[u]], tin[
71             u]+1));
72         u = par[head[u]];
73     }
74     if (dep[u] > dep[v]) swap(u, v);
75     ans = seg.merge(ans, seg.query(tin[u]+1, tin[v]+1));
76     return ans;
77 }
78 segtree::node query_subtree(int u) {
79     // query all edges in subtree of u
80     return seg.query(tin[u] + 1, tout[u]);
81 }

```

5.4 HLD - Heavy light decomposition

If you need to compute a function on a path in a tree and need to support value updates on nodes, HLD is the way. Pre-computation: $\mathcal{O}(n)$ Queries: $\mathcal{O}(\log^2 n)$

OBS: this implementation uses the same segtree as this notebook, with 0-indexing and open-closed interval convention. Ideally, just change the segtree to change the computed function, the HLD struct remains the same. OBS2: this template also supports mass updates (path/subtree) and subtree queries.

```

1 struct HLD {
2     int n, timer = 0;
3     vvi adj;
4     vi par, dep, sz, heavy, head, val, tin, tout;
5     segtree seg;
6     void init(int _n, vvi& _val, vvi& _adj) {
7         n = _n;
8         adj = _adj;
9         val = _val;
10        par = heavy = vi(n,-1);
11        dep = sz = head = tin = tout = vi(n);
12        timer = 0;

```

```

13        dfs1(0); dfs2(0,0);
14        vi linear(n);
15        for (int u = 0; u < n; u++)
16            linear[tin[u]] = val[u];
17        seg.init(linear);
18    }
19    int dfs1(int u) {
20        sz[u] = 1;
21        int max_sz = 0;
22        for (int v : adj[u]) {
23            if (v == par[u]) continue;
24            par[v] = u;
25            dep[v] = dep[u] + 1;
26            int size = dfs1(v);
27            sz[u] += size;
28            if (size > max_sz) {
29                max_sz = size;
30                heavy[u] = v;
31            }
32        }
33        return sz[u];
34    }
35    void dfs2(int u, int h) {
36        tin[u] = timer++;
37        head[u] = h;
38        if (heavy[u] != -1)
39            dfs2(heavy[u], h);
40        for (int v : adj[u]) {
41            if (v != par[u] && v != heavy[u])
42                dfs2(v, v);
43        }
44        tout[u] = timer;
45    }
46    void update(int u, int val) {
47        seg.set(tin[u], val);
48    }
49    void rangeUpdate(int u, int v, int x) {
50        while (head[u] != head[v]) {
51            if (dep[head[u]] < dep[head[v]]) swap(u, v);
52            seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
53            u = par[head[u]];
54        }
55        if (dep[u] > dep[v]) swap(u, v);
56        seg.rangeUpdate(tin[u], tin[v] + 1, x);
57    }
58    void update_subtree(int u, int x) {
59        seg.rangeUpdate(tin[u], tout[u], x);
60    }
61    segtree::node query(int u, int v) {
62        segtree::node ans;
63        while (head[u] != head[v]) {
64            if (dep[head[u]] < dep[head[v]])
65                swap(u, v);
66            ans = seg.merge(ans, seg.query(tin[head[u]], tin[
67                u]+1));
68            u = par[head[u]];
69        }
70        if (dep[u] > dep[v]) swap(u, v);
71        ans = seg.merge(ans, seg.query(tin[u], tin[v]+1));
72        return ans;
73    }
74    segtree::node query_subtree(int u){
75        return seg.query(tin[u], tout[u]);
76    }

```


5.5 LCA - RMQ

Pre-computation: $\mathcal{O}(n \log n)$

Queries: $\mathcal{O}(1)$

```
1 struct LCARMQ {
2     SparseTable st;
3     vi tin, dep, et_nodes, et_deps;
4     LCARMQ(vvi &adj){
5         int n = adj.size(), timer = 0;
6         tin = dep = vi(n);
7         et_nodes.reserve(2*n);
8         et_deps.reserve(2*n);
9         auto dfs = [&](auto &&dfs, int u, int p) -> void {
10             et_nodes.push_back(u);
11             et_deps.push_back(dep[u]);
12             tin[u] = timer++;
13             for (int v : adj[u]) if (v != p) {
14                 dep[v] = dep[u]+1;
15                 dfs(dfs,v,u);
16                 et_nodes.push_back(u);
17                 et_deps.push_back(dep[u]);
18             }
19             timer++;
20         };
21         dfs(dfs,0,0);
22         st.build(et_deps);
23     }
24     int get(int u, int v){
25         int tu = tin[u], tv = tin[v];
26         if (tu > tv) swap(tu,tv);
27         auto [val,id] = st.min(tu,tv);
28         return et_nodes[id];
29     }
30 };
```

5.6 LCA - binary lifting

Pre-computation: $\mathcal{O}(n \log n)$ Queries: $\mathcal{O}(\log n)$ OBS: just call `dfs(root)` before starting queries.

```
1 vvi adj, up;
2 vi tin, tout;
3 int timer = 0;
4 void dfs(int u, int p){
5     tin[u] = timer++;
6     for (auto v : adj[u]){
7         if (v == p) continue;
8         up[v][0] = u;
9         for (int dist = 1; dist < LOGN; dist++){
10             up[v][dist] = up[up[v][dist-1]][dist-1];
11         }
12         dfs(v);
13     }
14     tout[u] = timer++;
15 }
16 int isAncestor(int u, int v){
17     return tin[u] <= tin[v] && tout[v] <= tout[u];
18 }
19 int lca(int u, int v){
20     if (isAncestor(u,v)) return u;
21     if (isAncestor(v,u)) return v;
22     for (int dist = LOGN-1; dist >= 0; dist--){
23         if (!isAncestor(up[u][dist],v)) u = up[u][dist];
24     }
25     return up[u][0];
```

26 };

6 Problemas clássicos

6.1 2SAT

Struct for solving 2SAT problems that supports many types of boolean expressions. To add a negated literal use `u`

```
1 // para adicionar negacao usar ~u
2 // Ex: a clausula (a v !b) se traduz para add_or(a,~b)
3 struct TwoSatSolver {
4     int n;
5     vvi adj, adjT;
6     vector<bool> vis, assignment;
7     vi ts, scc;
8     void build(int _n){
9         n = 2*_n;
10        adj.assign(n,vi());
11        adjT.assign(n,vi());
12    }
13    int get(int u){
14        if (u < 0) return 2*(~u)+1;
15        else return 2*u;
16    }
17    void add_impl(int u, int v){ // u -> v
18        u = get(u), v = get(v);
19        adj[u].push_back(v);
20        adjT[v].push_back(u);
21        adj[v^1].push_back(u^1);
22        adjT[u^1].push_back(v^1);
23    }
24    void add_or(int u, int v){ // u || v
25        add_impl(~u, v);
26    }
27    void add_and(int u, int v){ // u && v
28        add_or(u,u); add_or(v,v);
29    }
30    void add_xor(int u, int v){ // u ^ v (u!=v)
31        add_impl(u, ~v);
32        add_impl(~u, v);
33    }
34    void add_equals(int u, int v){ // u == v
35        add_impl(u, v);
36        add_impl(v, u);
37    }
38    void tssort(int u){
39        vis[u] = true;
40        for (int v : adj[u])
41            if (!vis[v]) tssort(v);
42        ts.push_back(u);
43    }
44    void dfs(int u, int c){
45        scc[u] = c;
46        for (int v : adjT[u])
47            if (!scc[v]) dfs(v,c);
48    }
49    pair<bool, vector<bool>> solve(){
50        ts.clear();
51        vis.assign(n, false);
52        for (int i = 0; i < n; i++)
53            if (!vis[i]) tssort(i);
54        reverse(ts.begin(), ts.end());
55        scc.assign(n, 0);
56        int c = 0;
57        for (int u : ts)
```

```
58         if (!scc[u]) dfs(u,++c);
59        assignment.assign(n/2, false);
60        for (int i = 0; i < n; i += 2){
61            if (scc[i] == scc[i+1]) return {false, {}};
62            assignment[i/2] = scc[i] > scc[i+1];
63        }
64        return {true, assignment};
65    }
66 };
```

6.2 Next Greater Element

One of the classic stack applications. Easy to translate to lower, leq or geq, just change the comparator of the while.

```
1 vi ngt(n, n);
2 stack<ii> st;
3 for (int i = 0; i < n; i++){
4     while (!st.empty() && st.top().first < h[i]){
5         ngts[st.top().second] = i;
6         st.pop();
7     }
8     st.emplace(h[i], i);
9 }
```

7 Strings

7.1 Aho-Corasick

```
1 const int MAX = 1e6;
2 namespace aho {
3     int next[MAX][26];
4     int link[MAX], sz = 0, freq[MAX];
5     vi leaves[MAX], order;
6     void insert(const string& pat, int id) {
7         int u = 0;
8         for (char c : pat) {
9             int x = c - 'a';
10            if (!next[u][x]) next[u][x] = ++sz;
11            u = next[u][x];
12        }
13        leaves[u].push_back(id);
14    }
15    void build() {
16        queue<int> q; q.push(0);
17        link[0] = -1;
18        while (!q.empty()) {
19            int u = q.front(); q.pop();
20            order.push_back(u);
21            for (int x = 0; x < 26; x++) {
22                int v = next[u][x];
23                if (!v) continue;
24                int l = link[u];
25                while (l != -1 && !next[l][x]) l = link[l];
26                link[v] = l == -1 ? 0 : next[l][x];
27                q.push(v);
28            }
29        }
30    }
31    vi query(const string& txt, int num_patterns) {
32        vi ans(num_patterns);
33        int u = 0;
34        for (char c : txt){
35            int x = c - 'a';
```

```

36     while (u != -1 && !next[u][x]) u = link[u];
37     u = u == -1 ? 0 : next[u][x];
38     freq[u]++;
39 }
40 for (int i = (int)order.size() - 1; i >= 0; i--) {
41     int u = order[i];
42     if (link[u] != -1) freq[link[u]] += freq[u];
43 }
44 for (int i = 1; i <= sz; i++)
45     for (int id : leaves[i])
46         ans[id] += freq[i];
47 return ans;
48 }
49 }

```

7.2 Hashing

Creation time: $\mathcal{O}(n)$ Access time: $\mathcal{O}(1)$ Space: $\mathcal{O}(n)$

```

1 namespace DoubleHashing {
2     const int MOD1 = 2147483647, MOD2 = 2147483629;
3     using mint1 = mint<MOD1>;
4     using mint2 = mint<MOD2>;
5     vector<mint1> pmod1{1}, invpmod1{1};
6     vector<mint2> pmod2{1}, invpmod2{1};
7     const int MAX_VAL = 256;
8     const int OFFSET = 'a' - 1;
9     mt19937 rng(chrono::steady_clock::now().
10         time_since_epoch().count());
11     const mint1 p1 = uniform_int_distribution<int>(
12         MAX_VAL+1, MOD1-1)(rng);
13     const mint2 p2 = uniform_int_distribution<int>(
14         MAX_VAL+1, MOD2-1)(rng);
15     const mint1 inv1 = mint1::inv(p1);
16     const mint2 inv2 = mint2::inv(p2);
17     void extend(int n){
18         while(pmod1.size() <= n){
19             pmod1.push_back(pmod1.back()*p1);
20             pmod2.push_back(pmod2.back()*p2);
21             invpmod1.push_back(invpmod1.back()*inv1);
22             invpmod2.push_back(invpmod2.back()*inv2);
23         }
24     }
25     struct Hash {
26         mint1 h1;
27         mint2 h2;
28         int len = 0;
29         Hash apply(int x) const {
30             extend(len+1);
31             return {
32                 h1+pmod1[len]*(x-OFFSET),
33                 h2+pmod2[len]*(x-OFFSET),
34                 len+1
35             };
36         }
37     };
38     bool operator<(const Hash &o) const {
39         if (h1.val != o.h1.val) return h1.val < o.h1.val;
40         if (h2.val != o.h2.val) return h2.val < o.h2.val;
41         return len < o.len;
42     }
43     bool operator==(const Hash &o) const {
44         return h1.val == o.h1.val && h2.val == o.h2.val
45             && len == o.len;
46     }
47 }
48 }

```

```

47     pref.reserve(n+1);
48     pref.emplace_back();
49     for (auto c : s)
50         pref.push_back(pref.back().apply(c));
51 }
52 Hash get(int l, int r) const {
53     l++, r++;
54     mint1 h1 = (pref[r].h1 - pref[l-1].h1)*invpmod1[l-1];
55     mint2 h2 = (pref[r].h2 - pref[l-1].h2)*invpmod2[l-1];
56     return {h1, h2, r-l+1};
57 }
58 };
59 };

```

7.3 KMP

```

1 vi compute_lps(const string &pat){
2     int m = pat.length();
3     vi lps(m);
4     int len = 0;
5     for (int i = 1; i < m; i++){
6         while(len > 0 && pat[i] != pat[len])
7             len = lps[len-1];
8         if (pat[i] == pat[len]) len++;
9         lps[i] = len;
10    }
11    return lps;
12 }
13 // find all occurrences (simpler version)
14 vi kmp_search(const string &txt, const string &pat){
15     int n = txt.length(), m = pat.length();
16     vi lps = compute_lps(pat + '#' + txt);
17     vi occurrences;
18     for (int i = 0; i < n+m+1; i++){
19         if (lps[i] == pat.length())
20             occurrences.push_back(i-m*2);
21     }
22     return occurrences;
23 }
24 }
25 vi find_borders(const string &s){
26     vi lps = compute_lps(s);
27     int i = s.length()-1;
28     vi ans;
29     while (lps[i] > 0){
30         ans.push_back(lps[i]);
31         i = lps[i]-1;
32     }
33     reverse(ans.begin(), ans.end());
34     return ans;
35 }

```

7.4 Manacher

Computes the radius of the longest palindrome centered at every position in linear time, for even sized palindromes, center is the right index.

```

1 // ex:   a b a b b a c a
2 // odd:  1 2 2 1 1 1 2 1
3 // even:  0 0 0 0 2 0 0 0
4 struct Manacher {
5     vi odd, even;
6     int count = 0;

```

```

7 Manacher (const string &s) {
8     int n = s.size();
9     odd = even = vi(n);
10    for (int i = 0, l = 0, r = -1; i < n; i++) {
11        int k = i > r ? 1 : min(odd[l+r-i], r-i+1);
12        while (i-k >= 0 && i+k < n && s[i-k] == s[i+k]) k
13            ++;
14        odd[i] = k;
15        if (i+k-1 > r) l = i-k+1, r = i+k-1;
16    }
17    for (int i = 0, l = 0, r = -1; i < n; i++) {
18        int k = i > r ? 0 : min(even[l+r-i+1], r-i+1);
19        while (i-k-1 >= 0 && i+k < n && s[i-k-1] == s[i+k
20            ]) k++;
21        even[i] = k;
22        if (i+k-1 > r) l = i-k, r = i+k-1;
23    }
24    count = 0;
25    for (int i = 0; i < n; i++) count += odd[i]+even[i
26        ];
27 }
28 // query if substring s[l..r] is a palindrome
29 bool is_palindrome(int l, int r) {
30     int len = r-l+1, center = l+len/2;
31     if (len&1) return odd[center] > len/2;
32     return even[center] >= len/2;
33 }
34 };

```

7.5 Suffix Array

Time: $\mathcal{O}(n \log n)$ Space: $\mathcal{O}(n)$

```

1 struct SuffixArray {
2     vi sa, lcp;
3     SuffixArray(vi s, int lim = 256) {
4         s.push_back(0);
5         int n = s.size(), k = 0, a, b;
6         vi x(all(s)), y(n), ws(max(n, lim));
7         sa = lcp = y, iota(all(sa), 0);
8         for (int j=0, p=0; p<n; j = max(1ll, 2*j), lim=p) {
9             p = j, iota(all(y), n - j);
10            for (int i = 0; i < n; i++)
11                if (sa[i] >= j) y[p++] = sa[i]-j;
12            fill(all(ws), 0);
13            for (int i = 0; i < n; i++) ws[x[i]]++;
14            for (int i = 1; i <= lim; i++) ws[i] += ws[i-1];
15            for (int i = n; i--;) sa[--ws[x[i]]] = i;
16            swap(x, y), p = 1, x[sa[0]] = 0;
17            for (int i = 1; i < n; i++) {
18                a = sa[i-1], b = sa[i];
19                x[b] = (y[a]==y[b]&&y[a+j]==y[b+j]) ? p-1:p++;
20            }
21        }
22        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
23            for (k&&k--, j=sa[x[i]-1]; s[i+k]==s[j+k]; k++);
24    }
25 };

```

7.6 Suffix Automaton

```

1 #define ALPHABET 26
2 struct SAM {
3     struct State {
4         int len = 0, link = -1;

```

```

5   int cnt = 0, first_occ = -1;
6   int next[ALPHABET];
7   State() { fill(next, next + ALPHABET, -1); }
8   };
9   vector<State> st;
10  int last;
11  SAM(const string &s) {
12      st.reserve(2*s.length());
13      st.push_back(State());
14      last = 0;
15      for (int i = 0; i < s.size(); i++) extend(s[i], i);
16      calc_cnt();
17  }
18  void extend(char c, int id) {
19      int cur = st.size();
20      st.push_back(State());
21      st[cur].len = st[last].len + 1;
22      st[cur].first_occ = id;
23      st[cur].cnt = 1;
24      int p = last;
25      while (p != -1 && st[p].next[c-'a'] == -1) {
26          st[p].next[c-'a'] = cur;
27          p = st[p].link;
28      }
29      if (p == -1) {
30          st[cur].link = 0;
31          last = cur;
32          return;
33      }
34      int q = st[p].next[c-'a'];
35      if (st[p].len + 1 == st[q].len) {
36          st[cur].link = q;
37          last = cur;
38          return;
39      }
40      int clone = st.size();
41      st.push_back(st[q]);
42      st[clone].len = st[p].len + 1;
43      st[clone].cnt = 0;
44      while (p != -1 && st[p].next[c-'a'] == q) {
45          st[p].next[c-'a'] = clone;
46          p = st[p].link;
47      }
48      st[q].link = st[cur].link = clone;
49      last = cur;
50  }
51  void calc_cnt() {
52      int sz = st.size();
53      int mx = 0;
54      for (auto &s : st) mx = max(mx, s.len);
55      vi c(mx+1);
56      vi nodes(sz);
57      for (int i = 0; i < sz; i++) c[st[i].len]++;
58      for (int i = 1; i <= mx; i++) c[i] += c[i-1];
59      for (int i = 0; i < sz; i++) nodes[--c[st[i].len]]
60          = i;
61      for (int i = sz-1; i >= 0; i--) {
62          int u = nodes[i];
63          if (st[u].link != -1) st[st[u].link].cnt += st[u].cnt;
64      }
65  }
66  string longest_common_substring(const string &t) {
67      int v = 0, l = 0, len = 0, pos = 0;
68      for (int i = 0; i < t.length(); i++) {
69          int c = t[i] - 'a';
70          while (v != 0 && st[v].next[c] == -1) {
71              v = st[v].link;
72              l = st[v].len;

```

```

73      if (st[v].next[c] != -1) {
74          v = st[v].next[c];
75          l++;
76      }
77      if (l > len) {
78          len = l;
79          pos = i;
80      }
81  }
82  if (len == 0) return "";
83  return t.substr(pos-len+1, len);
84  }
85  int count_occurrences(const string &t) {
86      int cur = 0;
87      for (char c : t) {
88          if (st[cur].next[c-'a'] == -1) return 0;
89          cur = st[cur].next[c-'a'];
90      }
91      return st[cur].cnt;
92  }
93  int first_occurrence(const string &t) {
94      int cur = 0;
95      for (char c : t) {
96          if (st[cur].next[c-'a'] == -1) return -1;
97          cur = st[cur].next[c-'a'];
98      }
99      return st[cur].first_occ - t.length() + 1;
100  }
101  ll distinct_substrings() {
102      ll ans = 0;
103      for (int i = 1; i < st.size(); i++)
104          ans += st[i].len - st[st[i].link].len;
105      return ans;
106  }
107  vector<ll> distinct_substrings_perlen(int n) {
108      vector<ll> diff(n+2);
109      for (int i = 1; i < st.size(); i++) {
110          int l = st[st[i].link].len + 1;
111          int r = st[i].len;
112          diff[l]++; diff[r+1]--;
113      }
114      vector<ll> ans(n+1);
115      ans[0] = diff[0];
116      for (int i = 1; i <= n; i++) {
117          ans[i] = ans[i-1] + diff[i];
118      }
119      return ans;
120  }
121  vector<ll> dp;
122  void calc_paths(int u) {
123      if (dp[u] != -1) return;
124      dp[u] = 1;
125      for (int c = 0; c < ALPHABET; c++) {
126          int v = st[u].next[c];
127          if (v == -1) continue;
128          calc_paths(v);
129          dp[u] += dp[v];
130      }
131  }
132  string find_kth(ll k) {
133      dp.assign(st.size(), -1);
134      calc_paths(0);
135      int u = 0;
136      string ans = "";
137      while (k > 0) {
138          for (int c = 0; c < ALPHABET; c++) {
139              int v = st[u].next[c];
140              if (v == -1) continue;
141              if (k <= dp[v]) {
142                  ans += 'a' + c;

```

```

143          u = v;
144          k--;
145          break;
146      } else k -= dp[v];
147  }
148  }
149  return ans;
150  }
151  void calc_paths_with_repetitions(int u) {
152      if (dp[u] != -1) return;
153      dp[u] = st[u].cnt;
154      for (int c = 0; c < ALPHABET; c++) {
155          int v = st[u].next[c];
156          if (v == -1) continue;
157          calc_paths_with_repetitions(v);
158          dp[u] += dp[v];
159      }
160  }
161  string find_kth_with_repetitions(ll k) {
162      dp.assign(st.size(), -1);
163      calc_paths_with_repetitions(0);
164      int u = 0;
165      string ans = "";
166      while (k > 0) {
167          for (int c = 0; c < ALPHABET; c++) {
168              int v = st[u].next[c];
169              if (v == -1) continue;
170              if (k <= dp[v]) {
171                  ans += 'a' + c;
172                  u = v;
173                  k -= st[v].cnt;
174                  break;
175              } else k -= dp[v];
176          }
177      }
178      return ans;
179  }
180  };

```

7.7 Z

$$z[i] := \max(k) | s[0..k-1] = s[i..i+k-1]$$

Time: $\mathcal{O}(n+m)$ Space: $\mathcal{O}(n+m)$

```

1 vi compute_z(const string &s) {
2     int n = s.length();
3     vi z(n);
4     int l = 0, r = 0;
5     for (int i = 1; i < n; i++) {
6         if (i <= r) z[i] = min(r-i+1, z[i-l]);
7         while (i+z[i] < n && s[z[i]] == s[i+z[i]])
8             z[i]++;
9         if (i + z[i] - 1 > r) {
10             l = i; r = i+z[i]-1;
11         }
12     }
13     return z;
14 }
15 vi find_occurrences(const string &txt, const string &pat) {
16     vi occurrences;
17     vi z = compute_z(pat + '#' + txt);
18     int n = txt.length(), m = pat.length();
19     for (int i = 0; i < n+m+1; i++) {
20         if (z[i] == m) occurrences.push_back(i-m-1);
21     }
22     return occurrences;
23 }

```

8 Math

8.1 Combinatorics (Pascal's Triangle)

Computes "n choose k". Requires factorials to be pre-computed. Time: $\mathcal{O}(\log ZAP)$

8.1.1 Combinatorial Analysis

Fundamental Counting Principles

- **Permutations:** The number of ways to arrange k items from a set of n distinct items.

$$P(n, k) = \frac{n!}{(n - k)!}$$

- **Combinations (Binomial Coefficient):** The number of ways to choose k items from a set of n distinct items, regardless of order.

$$\binom{n}{k} = C(n, k) = \frac{n!}{k!(n - k)!}$$

- **Combinations with Repetition (Stars and Bars):** The number of ways to choose k items of n types, allowing repetitions. Equivalently, the number of ways to distribute k identical balls into n distinct urns.

$$\binom{k + n - 1}{n - 1} = \binom{k + n - 1}{k}$$

Binomial Coefficient Properties and Pascal's Triangle

- **Pascal's Triangle**

$$\begin{array}{c} \binom{0}{0} \\ \binom{1}{0} \binom{1}{1} \\ \binom{2}{0} \binom{2}{1} \binom{2}{2} \\ \binom{3}{0} \binom{3}{1} \binom{3}{2} \binom{3}{3} \\ \binom{4}{0} \binom{4}{1} \binom{4}{2} \binom{4}{3} \binom{4}{4} \end{array}$$

- **Stifel's Relation:** Each element in Pascal's Triangle is the sum of the two elements immediately above it.

$$\binom{n}{k} = \binom{n - 1}{k} + \binom{n - 1}{k - 1}$$

- **Symmetry:** Elements of a row are symmetric with respect to the center. Choosing k elements is the same as choosing the $n - k$ elements to be left behind.

$$\binom{n}{k} = \binom{n}{n - k}$$

- **Row Sum:** The sum of all elements in row n of Pascal's Triangle (where the first row is $n = 0$) is equal to 2^n .

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

- **Hockey Stick Identity:** The sum of elements in a diagonal, starting at

$$\binom{r}{r}$$

and ending at

$$\binom{n}{r}$$

, is equal to the element in the next row and next column,

$$\binom{n + 1}{r + 1}$$

$$\sum_{i=r}^n \binom{i}{r} = \binom{n + 1}{r + 1}$$

- **Binomial Theorem:**

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$$

- **Vandermonde's Identity:**

$$\sum_{j=0}^k \binom{m}{j} \binom{n}{k-j} = \binom{m+n}{k}$$

The easiest way to understand the identity is through a counting problem. Imagine you have a committee with m men and n women. How many ways can you form a subcommittee of k people?

Way 1 Direct Counting

You have a total of $m+n$ people and need to choose k of them. The number of ways to do this is simply:

$$\binom{m+n}{k}$$

Way 2 Counting by Cases

We can divide the problem into cases, based on how many men (j) are chosen for the subcommittee.

Case 0: Choose 0 men and k women. The number of ways is

$$\binom{m}{0} \binom{n}{k}$$

.

Case 1: Choose 1 man and $k-1$ women. The number of ways is

$$\binom{m}{1} \binom{n}{k-1}$$

.

Case j : Choose j men and $k-j$ women. The number of ways is

$$\binom{m}{j} \binom{n}{k-j}$$

.

Other Important Concepts

- **Catalan Numbers:** A sequence of natural numbers that occurs in various counting problems (e.g., number of binary trees, balanced parenthesis expressions).

$$C_n = \binom{2n}{n} - \binom{2n}{n+1} = \frac{1}{n+1} \binom{2n}{n}$$

A commonly used combinatorial proof for the Catalan numbers involves counting the number of lattice (grid) paths from $(0,0)$ to (n,n) that do not cross above the diagonal $y = x$. Each such path consists of n rightward steps and n upward steps, and the Catalan number counts the number of these "Dyck paths" that never go above the diagonal.

- **Stirling Numbers of the Second Kind:** The number of ways to partition a set of n labeled objects into k non-empty unlabeled subsets. Denoted by $S(n, k)$ or

$$\{n \ k\}$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

The Stirling numbers of the second kind can also be computed recursively:

$$S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$$

with the boundary conditions:

$$S(0, 0) = 1; \quad S(n, 0) = 0 \text{ for } n > 0; \quad S(0, k) = 0 \text{ for } k > 0$$

- **Bell Number:** The Bell number B^n counts the total number of ways to partition a set of n labeled elements into any number (from 1 up to n) of non-empty, unlabeled subsets. It can also be written as a recurrence relation

$$B^n = \sum_{k=0}^n S(n, k)$$

- **Pigeonhole Principle:** If n items are put into m boxes, with $n > m$, then at least one box must contain more than one item.

8.2 Convolutions

8.2.1 AND convolution

$$c[k] = \sum_{i \& j = k} a[i] \cdot b[j]$$

```
1 vector<mint> and_conv(vector<mint> a, vector<mint> b){
2   int n = a.size(); // must be pow of 2
3   for (int j = 1; j < n; j <= 1) {
4     for (int i = 0; i < n; i++) {
5       if (i&j) {
6         a[i~j] += a[i];
7         b[i~j] += b[i];
8       }
9     }
10  }
11  for (int i = 0; i < n; i++) a[i] *= b[i];
12  for (int j = 1; j < n; j <= 1)
13    for (int i = 0; i < n; i++)
14      if (i&j) a[i~j] -= a[i];
15  return a;
16 }
```

8.2.2 GCD convolution

$$c[k] = \sum_{\gcd(i,j)=k} a[i] \cdot b[j]$$

```
1 vector<mint> gcd_conv(vi a, vi b){
2   int n = (int)max(a.size(), b.size());
3   a.resize(n); b.resize(n);
4   vector<mint> c(n);
5   for (int i = 1; i < n; i++) {
6     mint x = 0;
7     mint y = 0;
8     for (int j = i; j < n; j += i) {
9       x += a[j];
10      y += b[j];
11    }
12    c[i] = x*y;
13  }
14  for (int i = n-1; i >= 1; i--)
15    for (int j = 2*i; j < n; j += i)
16      c[i] -= c[j];
17  return c;
18 }
```

8.2.3 LCM convolution

$$c[k] = \sum_{\text{lcm}(i,j)=k} a[i] \cdot b[j]$$

```
1 vector<mint> lcm_conv(vi a, vi b){
2   int n = (int)max(a.size(), b.size());
3   a.resize(n); b.resize(n);
4   vector<mint> c(n), x(n), y(n);
5   for (int i = 1; i < n; i++) {
6     for (int j = i; j < n; j += i) {
7       x[j] += a[i];
8       y[j] += b[i];
9     }
10    c[i] = x[i]*y[i];
11  }
```

```
12  for (int i = 1; i < n; i++)
13    for (int j = 2 * i; j < n; j += i)
14      c[j] -= c[i];
15  return c;
16 }
```

8.2.4 OR convolution

$$c[k] = \sum_{i|j=k} a[i] \cdot b[j]$$

```
1 vector<mint> or_conv(vector<mint> a, vector<mint> b){
2   int n = a.size(); // must be pow of 2
3   for (int j = 1; j < n; j <= 1) {
4     for (int i = 0; i < n; i++) {
5       if (i&j) {
6         a[i] += a[i~j];
7         b[i] += b[i~j];
8       }
9     }
10  }
11  for (int i = 0; i < n; i++) a[i] *= b[i];
12  for (int j = 1; j < n; j <= 1)
13    for (int i = 0; i < n; i++)
14      if (i&j) a[i] -= a[i~j];
15  return a;
16 }
```

8.2.5 Subset Convolution

$$c[k] = \sum_{i|j=k, i \& j = 0} a[i] \cdot b[j]$$

Complexity: $\mathcal{O}(k^2 2^k)$, where k is the number of bits.

```
1 vector<mint> subset_conv(const vector<mint>& a, const
2   vector<mint>& b) {
3   int n = a.size(); // must be pow of 2
4   int k = __builtin_ctz(n);
5   vector<vector<mint>> fa(k+1, vector<mint>(n, 0));
6   vector<vector<mint>> fb(k+1, vector<mint>(n, 0));
7   vector<vector<mint>> fc(k+1, vector<mint>(n, 0));
8   for (int i = 0; i < n; i++) {
9     int pc = __builtin_popcount(i);
10    fa[pc][i] = a[i];
11    fb[pc][i] = b[i];
12  }
13  for (int pc = 0; pc <= k; pc++) {
14    for (int j = 1; j < n; j <= 1) {
15      for (int i = 0; i < n; i++) {
16        if (i & j) {
17          fa[pc][i] += fa[pc][i~j];
18          fb[pc][i] += fb[pc][i~j];
19        }
20      }
21    }
22  }
23  for (int i = 0; i < n; i++) {
24    for (int pc = 0; pc <= k; pc++) {
25      for (int j = 0; j <= pc; j++) {
26        fc[pc][i] += fa[j][i] * fb[pc-j][i];
27      }
28    }
29  }
```



```

27 }
28 }
29 for (int pc = 0; pc <= k; pc++) {
30     for (int j = 1; j < n; j <= 1) {
31         for (int i = 0; i < n; i++) {
32             if (i & j) fc[pc][i] -= fc[pc][i ^ j];
33         }
34     }
35 }
36 vector<mint> c(n);
37 for (int i = 0; i < n; i++) {
38     c[i] = fc[___builtin_popcount(i)][i];
39 }
40 return c;
41 }

```

8.2.6 XOR convolution

$$c[k] = \sum_{i \oplus j = k} a[i] \cdot b[j]$$

```

1 void fwht(vector<mint> &a, bool inv){
2     int n = a.size(); // must be pow of 2
3     for (int step = 1; step < n; step <= 1){
4         for (int i = 0; i < n; i += 2*step) {
5             for (int j = i; j < i+step; j++){
6                 auto u = a[j];
7                 auto v = a[j+step];
8                 a[j] = u+v;
9                 a[j+step] = u-v;
10            }
11        }
12    }
13    if (inv) for (auto &x : a) x /= n;
14 }
15 vector<mint> xor_conv(vector<mint> a, vector<mint> b){
16     int n = a.size();
17     fwht(a,0), fwht(b, 0);
18     for (int i = 0; i < n; i++) a[i] *= b[i];
19     fwht(a,1);
20     return a;
21 }

```

8.3 Extended Euclid

Time: $\mathcal{O}(\log n)$.

```

1 int extended_gcd(int a, int b, int &x, int &y) {
2     x = 1, y = 0;
3     int x1 = 0, y1 = 1;
4     while (b) {
5         int q = a / b;
6         tie(x, x1) = make_tuple(x1, x - q * x1);
7         tie(y, y1) = make_tuple(y1, y - q * y1);
8         tie(a, b) = make_tuple(b, a - q * b);
9     }
10    return a;
11 }

```

8.4 Factorization

Fast factorization using Pollard-Rho Time: $\mathcal{O}(n^{1/4})$

```

1 void factorize(u64 n, vector<u64>& factors) {
2     if (n <= 1) return;
3     if (miller_rabin(n)) {
4         factors.push_back(n);
5         return;
6     }
7     u64 divisor = pollard_rho(n);
8     factorize(divisor, factors);
9     factorize(n / divisor, factors);
10 }

```

8.5 Gaussian elimination

Solves system of n linear equations for m variables, or informs there is no solution. Template is adaptable to use mint if necessary. a should have the last column as the "b" values. Time: $\mathcal{O}(n^2m)$

```

1 pair<int, vector<ld>> gauss(vector<vector<ld>> a) {
2     const double eps = 1e-6;
3     int n = a.size(), m = a[0].size()-1;
4     vi where(m, -1);
5     for (int var = 0, eq = 0; var < m && eq < n; var++) {
6         int piv = eq;
7         for (int i=eq; i<n; ++i)
8             if (abs(a[i][var]) > abs(a[piv][var])) piv = i;
9         if (abs(a[piv][var]) < eps) continue;
10        for (int i = var; i <= m; i++)
11            swap(a[piv][i], a[eq][i]);
12        where[var] = eq;
13        for (int i = 0; i < n; i++) if (i != eq) {
14            ld c = a[i][var] / a[eq][var];
15            for (int j = var; j <= m; j++)
16                a[i][j] -= a[eq][j] * c;
17        }
18        eq++;
19    }
20
21    vector<ld> ans(m, 0);
22    for (int i = 0; i < m; i++) if (where[i] != -1)
23        ans[i] = a[where[i]][m] / a[where[i]][i];
24    for (int i = 0; i < n; i++) {
25        ld sum = 0;
26        for (int j = 0; j < m; j++)
27            sum += ans[j] * a[i][j];
28        if (abs(sum - a[i][m]) > eps) return {0, {}};
29    }
30
31    for (int i = 0; i < m; i++) if (where[i] == -1)
32        return {INF, ans};
33    return {1, ans};
34 }

```

8.6 Inclusion-Exclusion Principle

For a set of k primes, count numbers from 1 to n divisible by at least one prime.

For every non-empty subset of primes:

- If the subset has odd size, add $n/prod$.
- If the subset has even size, subtract $n/prod$.
- $prod$ is the product of the selected primes.

Here, $n/prod$ is the number of integers from 1 to n divisible by all selected primes.

Complexity: $\mathcal{O}(2^k k)$.

```

1 int n, ans = 0;
2 vi primes;
3 for (int mask = 1; mask < (1<<primes.size()); mask++){
4     int prod = 1;
5     for (int bit = 0; bit < primes.size(); bit++) {
6         if (mask>>bit&1) {
7             prod *= primes[bit];
8             if (prod > n) break;
9         }
10    }
11    if (___popcount(mask) & 1) ans += n/prod;
12    else ans -= n/prod;
13 }

```

8.7 Legendre's formula

Computes the largest power of a prime p in $n!$

```

1 int legendre(int n, int p){
2     int ans = 0;
3     while(n>0){ n/=p; ans += n; }
4     return ans;
5 }

```

8.8 Linear Sieve + Möbius Function

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1 \\ (-1)^k & \text{if } n \text{ is a product of } k \text{ distinct primes} \\ 0 & \text{if } p^2 \mid n \text{ for some prime } p \end{cases}$$

8.8.1 Applications

- **Inclusion-Exclusion:** Naturally generates $1, -1, 0$ coefficients for prime factor sets.
- **Möbius Inversion:** Converts $g(n) = \sum_{d|n} f(d)$ to $f(n) = \sum_{d|n} \mu(d)g(\frac{n}{d})$. Often reduces $\mathcal{O}(N)$ divisor sums to $\mathcal{O}(\sqrt{N})$.
- **Dirichlet Convolution:** $\mu * 1 = \epsilon$. Core to sub-linear prefix sum algorithms (e.g., DuSieve) yielding $\mathcal{O}(N^{2/3})$ time.

```

1 auto sieve = [](int n) -> pair<vi,vi> {
2     vi primes, mu(n+1);
3     mu[1] = 1;
4     vector<bool> is_prime(n+1, true);
5     is_prime[0] = is_prime[1] = false;
6     for (int i = 2; i <= n; ++i) {
7         if (is_prime[i]) {
8             primes.push_back(i);
9             mu[i] = -1;
10        }
11        for (int p : primes) {
12            if (i*p > n) break;
13            is_prime[i*p] = false;
14            if (i%p == 0) {
15                mu[i*p] = 0;
16                break;
17            }
18            mu[i*p] = -mu[i];
19        }
20    }
21    return {primes, mu};
22 };

```

8.9 Matrix template

A template for square matrices, used for solving linear recurrences with fast exponentiation, memory is on a 1D vector, but can be accessed with `A[i][j]` normally (custom `operator[]`)

```

1 template<typename T>
2 struct mat{
3     vector<T> m;
4     int n;
5     mat(int _n = 0, bool identity = false) : n(_n) {
6         m.resize(n*n);
7         if (!identity) return;
8         for (int i = 0; i < n; i++)
9             m[i*n+i] = 1;
10    }
11    mat& operator+=(const mat &o){
12        for (int i = 0; i < n; i++){
13            int ra = i*n;
14            for (int j = 0; j < n; j++){
15                m[ra+j] += o.m[ra+j];
16            }
17        }
18        return *this;
19    }
20    mat& operator-=(const mat& o){
21        for (int i = 0; i < n; i++){
22            int ra = i*n;
23            for (int j = 0; j < n; j++){
24                m[ra+j] -= o.m[ra+j];
25            }
26        }
27        return *this;
28    }
29    mat& operator*=(const mat& o){
30        vector<T> ans(n*n);
31        for (int i = 0; i < n; i++){
32            for (int k = 0; k < n; k++){
33                int ra = i*n, rb = k*n;
34                for (int j = 0; j < n; j++){
35                    ans[ra+j] += m[ra+k] * o.m[rb+j];
36                }
37            }
38        }
39    }
40    friend mat operator+(mat a, const mat& b){return a+=b;
41    };
42    friend mat operator-(mat a, const mat& b){return a-=b;
43    };
44    friend mat operator*(mat a, const mat& b){return a*=b;
45    };
46    T* operator[](int i){
47        return &m[i*n];
48    }
49    vector<T> operator*(const vector<T> &v){
50        vector<T> ans(n);
51        for (int i = 0; i < n; i++){
52            int ra = i*n;
53            for (int j = 0; j < n; j++){
54                ans[i] += m[ra+j]*v[j];
55            }
56            return ans;
57        }
58        static mat exp(mat b, ll e){
59            mat ans = mat(b.n,true);
60            while(e>0){
61                if(e&1) ans*=b;
62                b*=b;
63                e>>=1;
64            }
65            return ans;
66        };

```

```

37    }
38    }
39    this->m = ans;
40    return *this;
41    }
42    friend mat operator+(mat a, const mat& b){return a+=b;
43    };
44    friend mat operator-(mat a, const mat& b){return a-=b;
45    };
46    friend mat operator*(mat a, const mat& b){return a*=b;
47    };
48    T* operator[](int i){
49        return &m[i*n];
50    }
51    vector<T> operator*(const vector<T> &v){
52        vector<T> ans(n);
53        for (int i = 0; i < n; i++){
54            int ra = i*n;
55            for (int j = 0; j < n; j++){
56                ans[i] += m[ra+j]*v[j];
57            }
58            return ans;
59        }
60        static mat exp(mat b, ll e){
61            mat ans = mat(b.n,true);
62            while(e>0){
63                if(e&1) ans*=b;
64                b*=b;
65                e>>=1;
66            }
67            return ans;
68        };

```

8.10 Miller-Rabin primality test

Time: $\mathcal{O}(k \log n)$ (k is the amount of bases tested, i.e. 7)

```

1 using u64 = uint64_t;
2 using u128 = __uint128_t;
3 u64 modpow(u64 b, u64 e, u64 mod) {
4     u64 ans = 1;
5     for (b %= mod; e; e >>= 1, b = (u128)b * b % mod)
6         if (e & 1) ans = (u128)ans * b % mod;
7     return ans;
8 }
9 // if up to 32 bits, use {2, 7, 61}
10 u64 primes[] = {2, 325, 9375, 28178, 450775, 9780504,
11                1795265022};
12 bool miller_rabin(u64 n) {
13     if (n < 2) return false;
14     if (n % 2 == 0) return n == 2;
15     u64 d = n - 1;
16     int s = 0;
17     while (d % 2 == 0) d >>= 1, s++;
18     for (u64 a : primes) {
19         if (a%n == 0) continue;
20         u64 x = modpow(a, d, n);
21         if (x == 1 || x == n - 1) continue;
22         bool composite = true;
23         for (int r = 1; r < s; r++) {
24             x = (u128)x * x % n;
25             if (x == n - 1) { composite = false; break; }
26         }
27         if (composite) return false;
28     }
29     return true;
30 }

```

8.11 Mint

```

1 template<const int MOD>
2 struct mint {
3     using m = mint;
4     int val;
5     m(ll v=0){
6         ll x = v%MOD;
7         if (x<0) x+=MOD;
8         val = x;
9     }
10    m& operator+=(const m &o){
11        ll x = (ll)val + o.val;
12        if (x >= MOD) x -= MOD;
13        val = x;
14        return *this;
15    }
16    m& operator-=(const m &o){
17        ll x = (ll)val - o.val;
18        if (x < 0) x += MOD;
19        val = x;
20        return *this;
21    }
22    m& operator*=(const m &o){
23        val = (ll)val*o.val%MOD;
24        return *this;
25    }
26    m& operator/=(const m &o){
27        val = (ll)val*inv(o).val%MOD;
28        return *this;
29    }
30    friend m operator+(m a, const m &b) {return a+=b;}
31    friend m operator-(m a, const m &b) {return a-=b;}
32    friend m operator*(m a, const m &b) {return a*=b;}
33    friend m operator/(m a, const m &b) {return a/=b;}
34    static m power(m b, int e){
35        m ans = 1;
36        while(e>0){
37            if (e&1) ans *= b;
38            b*=b; e>>=1;
39        }
40        return ans;
41    }
42    static m inv(m n){ return power(n,MOD-2); }
43 };

```

8.12 Modular Inverse

If m is prime, can use binary exponentiation to compute a^{p-2} (Fermat's Little Theorem).

This code works for non-prime m , as long as it is coprime to a .

Time: $\mathcal{O}(\log m)$

```

1 int modInverse(int a, int m) {
2     int x, y;
3     int g = extendedGcd(a, m, x, y);
4     if (g != 1) return -1;
5     return (x % m + m) % m;
6 }

```

8.13 Number Theoretic Transform (NTT)

NTT is a fast algorithm (analogous to FFT) for polynomial multiplication modulo a special prime. It requires a prime modulus $p = c \cdot 2^k + 1$ (a "primitive root prime") and a primitive 2^k -th root of unity modulo p .

- **Prime Choices:** To use NTT, pick a modulus and a matching primitive root (see table below). For arbitrary moduli (e.g., $10^9 + 7$), multiply with several NTT-friendly primes and reconstruct with CRT (see `crt_multiply`).
- **Time Complexity:** $\mathcal{O}(n \log n)$ for polynomial multiplication.

8.13.1 NTT-Friendly Primes and Roots

NTT-friendly primes and their primitive roots:

- Mod: 998244353, Root: 3, Max N: 2^{23}
- Mod: 734003201, Root: 3, Max N: 2^{20}
- Mod: 167772161, Root: 3, Max N: 2^{25}
- Mod: 469762049, Root: 3, Max N: 2^{26}

Use the modulus as MOD and the root as ROOT when instantiating the NTT.

- For large/concrete moduli, see `crt_multiply` in the code for a multi-modulus solution with Chinese Remainder Theorem (CRT).

```
1 namespace NTT{
2   template<typename T, ll MOD, ll ROOT>
3   void transform(vector<T>& a, bool invert) {
4     int n = a.size();
5
6     for (int i = 1, j = 0; i < n; i++) {
7       int bit = n >> 1;
8       for (; j & bit; bit >>= 1) j ^= bit;
9       j ^= bit;
10      if (i < j) swap(a[i], a[j]);
11    }
12
13    for (int len = 2; len <= n; len <= 1) {
14      T wlen = T::power(ROOT, (MOD - 1) / len);
15      if (invert) wlen = T::inv(wlen);
16      for (int i = 0; i < n; i += len) {
17        T w = 1;
18        for (int j = 0; j < len / 2; j++) {
19          T u = a[i + j], v = a[i + j + len / 2] * w;
20          a[i + j] = u + v;
21          a[i + j + len / 2] = u - v;
22          w *= wlen;
23        }
24      }
25    }
26
27    if (invert) {
28      T n_inv = T::inv(n);
```

```
29     for (T& x : a) x *= n_inv;
30   }
31 }
32
33 template<typename T, ll MOD, ll ROOT>
34 vector<ll> multiply(const vector<ll>& a, const vector
35 <ll>& b) {
36   vector<T> fa(a.begin(), a.end()), fb(b.begin(), b.
37   end());
38   int n = 1;
39   while (n < a.size() + b.size()) n <= 1;
40   fa.resize(n);
41   fb.resize(n);
42
43   transform<T, MOD, ROOT>(fa, false);
44   transform<T, MOD, ROOT>(fb, false);
45
46   for (int i = 0; i < n; i++) fa[i] *= fb[i];
47
48   transform<T, MOD, ROOT>(fa, true);
49
50   vector<ll> result(n);
51   for (int i = 0; i < n; i++) result[i] = fa[i].val;
52   return result;
53 }
54
55 vector<ll> crt_multiply(const vector<ll>& a, const
56 vector<ll>& b) {
57   const ll mod1 = 998244353;
58   const ll root1 = 3;
59   using mint1 = mint<mod1>;
60   vector<ll> ans1 = NTT::multiply<mint1, mod1, root1>
61   >(a, b);
62
63   const ll mod2 = 1004535809;
64   const ll root2 = 3;
65   using mint2 = mint<mod2>;
66   vector<ll> ans2 = NTT::multiply<mint2, mod2, root2>
67   >(a, b);
68
69   int ans_size = a.size() + b.size() - 2;
70   ll M1_inv_M2 = mint<mod2>::inv(mod1).val;
71
72   vector<ll> final_result(ans_size + 1);
73   for (int i = 0; i <= ans_size; ++i) {
74     ll v1 = ans1[i];
75     ll v2 = ans2[i];
76     ll k = ((v2 - v1 + mod2) % mod2 * M1_inv_M2) %
77     mod2;
78     final_result[i] = v1 + k * mod1;
79   }
80   return final_result;
81 }
```

8.14 Pollard-Rho

Returns a non-trivial factor of n , used for fast factorization
Time: $\mathcal{O}(n^{1/4})$

```
1 u64 pollard_rho(u64 n) {
2   if (n % 2 == 0) return 2;
3   if (miller_rabin(n)) return n;
4   static mt19937_64 rng(1337); // change to chrono
5   u64 x = 0, y = 0, c = 0, prd = 1, d = 1;
6   auto f = [&](u64 v) { return ((u128)v*v + c)%n; };
7   while (d == 1 || d == n) {
```

```
8     x = y = rng() % (n - 2) + 2;
9     c = rng() % (n - 1) + 1;
10    d = 1;
11    for (int step = 1; d == 1; step <= 1) {
12      x = y;
13      for (int i = 0; i < step; ++i) y = f(y);
14      for (int k = 0; k < step && d == 1; k += 128) {
15        prd = 1;
16        int limit = min(step - k, 128);
17        for (int i = 0; i < limit; ++i) {
18          y = f(y);
19          prd = (u128)prd * (x > y ? x - y : y - x)%n;
20        }
21        d = gcd(prd, n);
22      }
23    }
24    return d;
25  }
```

8.15 Euler's Totient

Returns the amount of numbers smaller than n that are coprime to n . Time: $\mathcal{O}(\sqrt{n})$

```
1 int phi(int n){
2   int ans = n;
3   for (int i = 2; i*i <= n; i++){
4     if (n%i == 0){
5       while(n%i == 0) n/=i;
6       ans -= ans/i;
7     }
8   }
9   if (n>1) ans -= ans/n;
10  return ans;
11 }
```

8.16 Xor Basis

Powerful for some weird xor problems, represents the linearly independent basis whose span is the set of elements you inserted into it.

```
1 #define D 60 // max bits
2 struct XorBasis {
3   ll a[D] = {0};
4   int sz = 0;
5   // Returns {represented_part, orthogonal_part}
6   pair<ll, ll> reduce(ll x) {
7     ll orig = x;
8     for (int i = D-1; i >= 0; i--)
9       if ((x>>i&1) && a[i]) x ^= a[i];
10    return {orig^x, x};
11  }
12  bool insert(ll x) {
13    for (int i = D-1; i >= 0; i--) {
14      if (!(x>>i&1)) continue;
15      if (!a[i]) {
16        a[i] = x; sz++;
17        return true;
18      }
19      x ^= a[i];
20    }
21    return false;
```

```

22 }
23 bool check(ll x) { return reduce(x).second == 0; }
24 ll max_xor(ll val = 0) {
25     for (int i = D-1; i >= 0; i--)
26         val = max(val, val^a[i]);
27     return val;
28 }
29 ll span_size() { return ll1 << sz; }
30 };

```

9 Geometry

9.1 Geometry

Always try to use integer coordinates. And avoid using dist for everything (hypot is slow AF), use dist2.

```

1 // heavily inspired by brunomaletta/biblioteca
2 using T = long long; // change to ld if necessary
3 const ld DINF = 2e18;
4 const ld pi = acos(-1.0);
5 const ld eps = 1e-9;
6 #define sq(x) ((x)*(x))
7 // -1 for negative, 0 for zero, 1 for positive
8 int sgn(T x) {
9     if constexpr (is_floating_point_v<T>)
10         return (x > eps) - (x < -eps);
11     return (x > 0) - (x < 0);
12 }
13 struct pt {
14     T x, y;
15     pt(T x_ = 0, T y_ = 0) : x(x_), y(y_) {}
16     bool operator<(const pt p) const {
17         if (sgn(x-p.x) != 0) return sgn(x-p.x) < 0;
18         return sgn(y-p.y) < 0;
19     }
20     bool operator==(const pt p) const {
21         return sgn(x-p.x) == 0 && sgn(y-p.y) == 0;
22     }
23     pt operator+(pt p) const {return pt(x+p.x, y+p.y);}
24     pt operator-(pt p) const {return pt(x-p.x, y-p.y);}
25     pt operator*(T c) const {return pt(x*c, y*c);}
26     pt operator/(T c) const {return pt(x/c, y/c);}
27     T operator*(pt p) const {return x*p.x + y*p.y;}
28     T operator^(pt p) const {return x*p.y - y*p.x;}
29     friend istream& operator >> (istream& in, pt& p) {
30         return in >> p.x >> p.y;
31     }
32 };
33 struct line {
34     pt p, q;
35     line() {}
36     line(pt p_, pt q_) : p(p_), q(q_) {}
37     friend istream& operator >> (istream& in, line& r) {
38         return in >> r.p >> r.q;
39     }
40 };
41
42 // --- INTEGER SAFE OPERATIONS --- //
43
44 ld dist(pt p, pt q) {
45     return hypot((ld)(p.y - q.y), (ld)(p.x - q.x));
46 }
47 T dist2(pt p, pt q) {return sq(p.x-q.x) + sq(p.y-q.y);}
48 ld norm(pt v) { return dist(pt(0, 0), v); }
49 // Angle with +x axis in [0, 2*pi)

```

```

50 ld angle(pt v) {
51     ld ang = atan2((ld)v.y, (ld)v.x);
52     return ang < 0 ? ang + 2*pi : ang;
53 }
54 // 2x signed area of triangle p-q-r (>0 if ccw)
55 T sarea2(pt p, pt q, pt r) { return (q-p)^(r-q); }
56 // True if p, q, r are collinear
57 bool col(pt p, pt q, pt r) {return sgn(sarea2(p,q,r))==0;}
58 // True if r is strictly to the left of line p->q
59 bool ccw(pt p, pt q, pt r) {return sgn(sarea2(p,q,r))>0;}
60 bool isvertical(line r) {return sgn(r.p.x-r.q.x) == 0;}
61 // True if point p lies on segment r
62 bool isinseg(pt p, line r) {
63     pt a = r.p - p, b = r.q - p;
64     return sgn(a^b) == 0 && sgn(a * b) <= 0;
65 }
66 bool interseg(line r, line s) {
67     if (isinseg(r.p, s) || isinseg(r.q, s) ||
68         isinseg(s.p, r) || isinseg(s.q, r)) return 1;
69     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
70         ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
71 }
72 ld disttoline(pt p, line r) {
73     return (ld)abs(sarea2(p, r.p, r.q)) / dist(r.p, r.q);
74 }
75 ld disttoseg(pt p, line r) {
76     if (sgn((r.q-r.p)*(p-r.p)) < 0) return dist(r.p,p);
77     if (sgn((r.p-r.q)*(p-r.q)) < 0) return dist(r.q,p);
78     return disttoline(p, r);
79 }
80
81 // --- POLYGONS (INTEGER SAFE) --- //
82
83 ld polarea(const vector<pt> &v) {
84     T ans = 0;
85     for (int i = 0; i < sz(v); i++)
86         ans += (v[i]^v[(i+1)%sz(v)]);
87     return abs((ld)ans / 2.0);
88 }
89 // Returns 0 if out, 1 if inside, 2 if on border
90 int inpol(const vector<pt> &v, pt p) {
91     int qt = 0;
92     for (int i = 0; i < sz(v); i++) {
93         if (p == v[i]) return 2;
94         int j = (i+1)%sz(v);
95         if (sgn(p.y-v[i].y)==0 && sgn(p.y-v[j].y)==0) {
96             if (sgn((v[i]-p)*(v[j]-p)) <= 0) return 2;
97             continue;
98         }
99         bool lower = sgn(v[i].y - p.y) < 0;
100         if (lower == (sgn(v[j].y - p.y) < 0)) continue;
101         auto t = (p-v[i])^(v[j]-v[i]);
102         if (sgn(t) == 0) return 2;
103         if (lower == (sgn(t) > 0)) qt += lower ? 1 : -1;
104     }
105     return qt != 0;
106 }
107 // Monotone chain convex hull - O(N log N)
108 // icol - include_collinear
109 vector<pt> convex_hull(vector<pt> v, bool icol = 0) {
110     sort(all(v));
111     v.erase(unique(all(v)), v.end());
112     int n = sz(v);
113     if (n <= 1) return v;
114     vector<pt> l, u;
115     auto bad = [&](pt a, pt b, pt c) {
116         if (icol) return sgn(sarea2(a, b, c)) < 0;
117         return sgn(sarea2(a, b, c)) <= 0;
118     };
119     for (int i = 0; i < n; i++) {

```

```

120         while (sz(l)>1 &&
121             bad(l[sz(l)-2], l[sz(l)-1], v[i]))
122             l.pop_back();
123         l.push_back(v[i]);
124     }
125     for (int i = n-1; i >= 0; i--) {
126         while (sz(u)>1 && bad(u[sz(u)-2], u[sz(u)-1], v[i]))
127             u.pop_back();
128         u.push_back(v[i]);
129     }
130     l.pop_back(); u.pop_back();
131     l.insert(l.end(), all(u));
132     return l;
133 }
134 // d^2 of closest pair - O(N log N)
135 T closest_pair(vector<pt> v) {
136     if (sz(v) <= 1) return 0;
137     sort(all(v));
138     vector<pt> t(sz(v));
139     auto go = [&](auto& go, int l, int r) -> T {
140         if (r-l <= 1) return DINF;
141         int m = l + (r-l)/2;
142         T mid = v[m].x;
143         T ans = min(go(go, l, m), go(go, m, r));
144         merge(begin(v)+l, begin(v)+m, begin(v)+m, begin(v)+
145             r,
146             begin(t), [](pt a, pt b){return sgn(a.y-b.y)
147                 < 0;});
148         copy(t.begin(), t.begin()+(r-l), v.begin()+l);
149         int k = 0;
150         for (int i = l; i < r; i++) {
151             if (sgn(sq(v[i].x - mid) - ans) >= 0) continue;
152             for (int j = k-1; j >= 0; j--) {
153                 if (sgn(sq(v[i].y-t[j].y)-ans) >= 0) break;
154                 ans = min(ans, dist2(v[i], t[j]));
155             }
156             t[k++] = v[i];
157         }
158         return ans;
159     };
160     return go(go, 0, sz(v));
161 }
162 struct convex_pol {
163     vector<pt> pol;
164     convex_pol(vector<pt> v) : pol(convex_hull(v)) {}
165     // O(log N) check if p is inside convex polygon
166     bool is_inside(pt p) {
167         if (pol.empty()) return false;
168         if (sz(pol) == 1) return p == pol[0];
169         int l = 1, r = sz(pol);
170         while (l < r) {
171             int m = l + (r - l) / 2;
172             if (ccw(p, pol[0], pol[m])) l = m+1;
173             else r = m;
174         }
175         if (l == 1) return isinseg(p, line(pol[0], pol[1]));
176         if (l == (int)sz(pol)) return false;
177         return !ccw(p, pol[l], pol[l-1]);
178     }
179     // O(log N) extreme vertex given custom comparator
180     int extreme(const function<bool(pt, pt)>& cmp) {
181         int n = sz(pol);
182         auto extr = [&](int i, bool& cur_dir) {
183             cur_dir = cmp(pol[(i+1)%n], pol[i]);
184             return !cur_dir && !cmp(pol[(i+n-1)%n], pol[i]);
185         };
186         bool last_dir, cur_dir;
187         if (extr(0, last_dir)) return 0;

```

```

187 int l = 0, r = n;
188 while (l+1 < r) {
189     int m = l + (r - l) / 2;
190     if (extr(m, cur_dir)) return m;
191     bool rel_dir = cmp(pol[m], pol[l]);
192     if ((!last_dir && cur_dir) ||
193         (last_dir==cur_dir && rel_dir==cur_dir)) {
194         l = m; last_dir = cur_dir;
195     } else r = m;
196 }
197 return l;
198 }
199 // O(log N) furthest vertex in direction of vector v
200 int max_dot(pt v) {
201     return extreme(
202         [&](pt p, pt q) { return sgn(p*v - q*v) > 0; }
203     );
204 }
205 // O(log N) indices of tangent vertices
206 // from point p strictly outside
207 pair<int, int> tangents(pt p) {
208     return {
209         extreme([&](pt q, pt r){ return ccw(p,r,q); })),
210         extreme([&](pt q, pt r){ return ccw(p,q,r); }));
211 }
212 };
213
214 // =====//
215 // --- FLOAT ONLY (REQUIRE 'using T = ld;') --- //
216 // =====//
217
218 // th in radians
219 pt rotate(pt p, ld th) {
220     return pt(
221         p.x*cos(th) - p.y*sin(th),
222         p.x*sin(th) + p.y*cos(th)
223     );
224 }
225 pt rotate90(pt p) { return pt(-p.y, p.x); }
226 // Scalar t such that
227 // r.p + t*(r.q - r.p) intersects line pt(0,0) -> v
228 ld get_t(pt v, line r) {
229     return (ld)(r.p~r.q) / (ld)((r.p-r.q)^v);
230 }
231 pt proj(pt p, line r) {
232     if (r.p == r.q) return r.p;
233     r.q = r.q - r.p; p = p - r.p;
234     return r.q * ((p~r.q) / (r.q~r.q)) + r.p;
235 }
236 pt inter(line r, line s) {
237     if (sgn((r.p - r.q) ^ (s.p - s.q)) == 0)
238         return pt(DINF, DINF);
239     pt rq = r.q - r.p;
240     return rq * get_t(rq, s) + r.p;
241 }
242 // Cuts polygon with line r leaving points
243 // strictly to the left of r
244 vector<pt> cut_polygon(vector<pt> v, line r) {
245     vector<pt> ans;
246     for (int j = 0; j < (int)sz(v); j++) {
247         if (ccw(r.p, r.q, v[j])) ans.push_back(v[j]);
248         if (sz(v) == 1) continue;
249         line s(v[j], v[(j+1)%sz(v)]);
250         pt p = inter(r, s);
251         if (isinseg(p, s)) ans.push_back(p);
252     }
253     ans.erase(unique(all(ans)), ans.end());
254     if (sz(ans) > 1 && ans.back() == ans[0])
255         ans.pop_back();
256     return ans;
257 }
258
259 struct circle {
260     pt c; ld r; // customizable to ll
261     circle(pt x, pt y, pt z) {
262         y = (x+y)/2.0; z = (x+z)/2.0;
263         c = inter(line(y, y + rotate90(x-y)),
264                 line(z, z + rotate90(x-z)));
265         r = dist(c, x);
266     }
267     circle(pt c, ld r) : c(c), r(r) {}
268 };
269 vector<pt> circ_line_inter(const line &l, circle &c) {
270     vector<pt> ans;
271     pt b = l.q - l.p, a = l.p - c.c;
272     ld A = b~b, B = a~b, C = a~a - c.r*c.r;
273     ld D = B*B - A*C;
274     if (sgn(D) < 0) return ans;
275     ans.push_back(c.c+a+b*(-B+sqrt(max((ld)0,D)))/A);
276     if (sgn(D) > 0)
277         ans.push_back(c.c+a+b*(-B-sqrt(max((ld)0,D)))/A);
278     return ans;
279 }
280 vector<pt> circ_inter(circle ca, circle cb) {
281     pt a = ca.c, b = cb.c;
282     ld r = ca.r, R = cb.r, d = dist(a, b);
283     vector<pt> ans;
284     if (sgn(d - (r+R)) > 0 ||
285         sgn(d + min(r, R) - max(r, R)) < 0) return ans;
286     ld x = (d*d - R*R + r*r) / (2.0*d);
287     ld y = sqrt(max((ld)0.0, r*r - x*x));
288     pt v = (b-a)/d;
289     ans.push_back(a + v*x + rotate90(v)*y);
290     if (sgn(y) > 0) ans.push_back(a+v*x-rotate90(v)*y);
291     return ans;
292 }
293 // --- POLYGON & LATTICE (INTEGER SAFE) --- //
294
295 // Centroid of a polygon. Area must be > 0.
296 pt centroid(const vector<pt>& v) {
297     pt c(0, 0); ld A = 0;
298     for (int i = 0; i < (int)sz(v); i++) {
299         ld cross = (v[i] ^ v[(i+1)%sz(v)]);
300         A += cross;
301         c = c + (v[i] + v[(i+1)%sz(v)]) * cross;
302     }
303     return c / (3.0 * A);
304 }
305 // Num of integer lattice points on polygon boundary
306 int boundary_pts(const vector<pt>& v) {
307     int b = 0;
308     for (int i = 0; i < (int)sz(v); i++) {
309         pt p = v[i], q = v[(i+1)%sz(v)];
310         b += gcd(abs((int)(p.x-q.x)), abs((int)(p.y-q.y)));
311     }
312     return b; // Obs: Interior pts I = (2*Area-B+2)/2
313 }
314 // Minkowski sum of two CONVEX polygons - O(N*M)
315 void reorder_pol(vector<pt>& p) {
316     int pos = 0;
317     for(int i = 1; i < (int)sz(p); i++) {
318         if(p[i].y < p[pos].y ||
319            (p[i].y==p[pos].y && p[i].x < p[pos].x)) pos = i;
320     }
321     rotate(p.begin(), p.begin() + pos, p.end());
322 }
323 vector<pt> minkowski(vector<pt> P, vector<pt> Q) {
324     reorder_pol(P); reorder_pol(Q);
325     P.push_back(P[0]); P.push_back(P[1]);
326     Q.push_back(Q[0]); Q.push_back(Q[1]);
327
328     vector<pt> ans;
329     int i = 0, j = 0;
330     while(i < (int)sz(P)-2 || j < (int)sz(Q)-2) {
331         ans.push_back(P[i] + Q[j]);
332         auto cross = (P[i+1] - P[i]) ^ (Q[j+1] - Q[j]);
333         if(cross >= 0 && i < (int)sz(P) - 2) i++;
334         if(cross <= 0 && j < (int)sz(Q) - 2) j++;
335     }
336     return ans;
337 }
338 // O(N) diameter of CONVEX pol by rotating calipers
339 ld diameter(vector<pt> p) {
340     if(sz(p) <= 1) return 0;
341     if(sz(p) == 2) return dist(p[0], p[1]);
342     ld ans = 0;
343     for(int i = 0, j = 1; i < (int)sz(p); i++) {
344         pt nxt = p[(i+1)%sz(p)];
345         while(sarea2(p[i], nxt, p[(j+1)%sz(p)]) >
346             sarea2(p[i], nxt, p[j])) j = (j+1)%sz(p);
347         ans = max({ans, dist(p[i],p[j]), dist(nxt,p[j])});
348     }
349     return ans;
350 }
351 // =====//
352 // --- FLOAT ONLY (REQUIRE 'using T = ld;') --- //
353 // =====//
354
355 struct halfplane {
356     pt p, pq; ld ang;
357     halfplane() {}
358     halfplane(pt a, pt b) : p(a), pq(b - a) {
359         ang = atan2((ld)pq.y, (ld)pq.x);
360     }
361     bool out(pt r) { return sgn(pq ^ (r - p)) < 0; }
362     bool operator<(const halfplane& e) const {
363         if(sgn(ang-e.ang) == 0) return sgn(pq^(e.p-p))<0;
364         return ang < e.ang;
365     }
366     pt inter(const halfplane& s) {
367         pt rq = s.p - p;
368         ld t = (ld)(s.pq ^ rq) / (ld)(pq ^ s.pq);
369         return p + pq * t;
370     }
371 };
372 // Half-Plane Intersection - O(N log N)
373 // Returns convex polygon bounding the intersection
374 vector<pt> hpi(vector<halfplane> H) {
375     sort(all(H));
376     deque<halfplane> q; deque<pt> ans;
377     for(int i = 0; i < (int)sz(H); i++) {
378         while(!ans.empty() && H[i].out(ans.back())) {
379             ans.pop_back(); q.pop_back();
380         }
381         while(!ans.empty() && H[i].out(ans.front())) {
382             ans.pop_front(); q.pop_front();
383         }
384         if(i>0 && sgn(H[i].ang-H[i-1].ang) == 0) continue;
385         q.push_back(H[i]);
386         if(sz(q) > 1)
387             ans.push_back(q.back().inter(q[sz(q)-2]));
388     }
389     while(!ans.empty() && q.front().out(ans.back())) {
390         ans.pop_back(); q.pop_back();
391     }
392     if(sz(q) < 3) return {};
393     ans.push_back(q.back().inter(q.front()));
394     return vector<pt>(all(ans));
395 }
396 // Minimum Enclosing Circle (Welzl's Algorithm)

```



```

397 // O(N) randomized
398 circle welzl(vector<pt> v) {
399     mt19937 rng(1337); // change to chrono
400     shuffle(all(v), rng);
401     circle c(pt(0,0),0);
402     for (int i = 0; i < sz(v); i++) {
403         if (dist(c.c, v[i]) <= c.r + eps) continue;
404         c = circle(v[i], 0);
405         for (int j = 0; j < i; j++) {
406             if (dist(c.c, v[j]) <= c.r + eps) continue;
407             c = circle((v[i]+v[j])/2.0, dist(v[i],v[j])/2.0);
408             for (int k = 0; k < j; k++) {
409                 if (dist(c.c, v[k]) <= c.r + eps) continue;
410                 c = circle(v[i], v[j], v[k]);
411             }
412         }
413     }
414     return c;
415 }
416
417 // --- COMPARATORS FOR SWEEP-LINE ALGORITHMS --- //
418
419 // Comparator for lines (assumes p < q)
420 bool operator<(const line& a, const line& b) {
421     pt v1 = a.q - a.p, v2 = b.q - b.p;
422     ld a1 = angle(v1), a2 = angle(v2);
423     if (sgn(a1 - a2) != 0) return a1 < a2;
424     return ccw(a.p, a.q, b.p);
425 }
426
427 bool operator==(const line& a, const line& b) {
428     return !(a < b) && !(b < a);
429 }
430
431 // Sweep-line state comparator
432 // (horizontal sweep, vertical sort)
433 struct cmp_sweepangle {
434     pt dir; // Set this globally before comparing
435     bool operator()(const line& a, const line& b) const {
436         return sgn(get_t(dir, a) - get_t(dir, b)) < 0;
437     }
438 };
439
440 if (a.p == b.p) return ccw(a.p, a.q, b.q);
441 if (sgn(a.p.x - a.q.x) != 0 &&
442     (sgn(b.p.x - b.q.x) == 0 || sgn(a.p.x - b.p.x) < 0))
443     return ccw(a.p, a.q, b.p);
444 return ccw(a.p, b.q, b.p);
445 }
446
447 // Sweep-angle state comparator
448 // (radial sweep, distance sort)
449 struct cmp_sweepangle {
450     pt dir; // Set this globally before comparing
451     bool operator()(const line& a, const line& b) const {
452         return sgn(get_t(dir, a) - get_t(dir, b)) < 0;
453     }
454 };
455
456 if (a.p == b.p) return ccw(a.p, a.q, b.q);
457 if (sgn(a.p.x - a.q.x) != 0 &&
458     (sgn(b.p.x - b.q.x) == 0 || sgn(a.p.x - b.p.x) < 0))
459     return ccw(a.p, a.q, b.p);
460 return ccw(a.p, b.q, b.p);
461 }
462
463 // Sweep-line state comparator
464 // (horizontal sweep, vertical sort)
465 struct cmp_sweepangle {
466     pt dir; // Set this globally before comparing
467     bool operator()(const line& a, const line& b) const {
468         return sgn(get_t(dir, a) - get_t(dir, b)) < 0;
469     }
470 };
471
472 if (a.p == b.p) return ccw(a.p, a.q, b.q);
473 if (sgn(a.p.x - a.q.x) != 0 &&
474     (sgn(b.p.x - b.q.x) == 0 || sgn(a.p.x - b.p.x) < 0))
475     return ccw(a.p, a.q, b.p);
476 return ccw(a.p, b.q, b.p);
477 }

```